# Coen's AI Notes and Links

(by several intelligences cooperating)

2026-11-11

# Table of contents


1 Introduction 1

I Onboarding 3

2 How to download 4

II Unsorted: Hot or Not? 6

3 Hot or not 7

4 OpenAI agents hack HuggingFace 9

5 The Risen Framework 10

6 AI BluePrint 11

7 Ponytail 12

8 SuperPowers 13

9 Mythos (June 2026) 14

10 NeurIPS 2026 15

**III AI Field Overview** **16**

**11 Throw some keywords** **17**

**12 AI Overview: AI, Machine Learning, Deep Learning, Generative AI** **18**

**13 Prediction 2025** **19**

**14 Geoffrey Hinton about Neural Networks** **20**

**15 How LLMs work** **21**

**16 GenAI** **23**
16.1 What is GenAI? . . . 23
16.2 GPT - Generative Pre-Trained Transformer . . . 24
16.3 Prompting . . . 24
16.4 Hallucinating . . . 25
16.5 RAG - Retrieval Augmented Generation . . . 25
16.6 Active Inference . . . 25
16.7 Running LLM's locally . . . 25
16.8 GenAI . . . 25
16.9 Some more sites, nice to play around with . . . 26

**17 Resources and References GenAI** **27**
17.1 How te mention that you did use AI? . . . 27
17.2 Blogs and articles . . . 27
17.3 Online Platforms . . . 27
17.4 (Short) Courses . . . 27
17.5 Code Repositories . . . 28
17.6 Community Resources . . . 28
17.7 Academic Papers: Modern Breakthroughs . . . 28

**18 No-Code / Low Code** **29**

**19 EU AI Act** **30**

**20 Popular Data Sources** **31**

**21 Guardrailing** **32**

**22 Transformers** **33**

**23 Multi Model** **34**

## IV AI Literacy (mostly GenAI) 35

**24 AI Literacy** **36**

**25 Can AI steal my data?** **37**

**26 Key Tools to Get Started** **38**

**27 Use AI as** **39**

**28 LLMs unplugged** **40**

**29 AI Jargon Buster** **41**

**30 Transcribing / Speech2Text** **42**

**31 Scenario: Summarize a Text** **43**

**32 Beyond the Prompt: The Power of AI Context** **44**
32.1 Advanced: Let the LLM Write the Prompts . . . . . 44
32.2 Context Engineering . . . . . 44

**33 Scenario: Replying to an Email** **45**

**34 Prompting for Everyone (Andrew Ng)** **46**

**35 Prompting: A.I.M.** **48**

**36 Some AI quirks** **51**

**37 AI Failures: When Image Context is Misleading** **52**

**38 Create an app with AI** **54**

**39 Scenario: A 17 Minute AI Workflow To Stand Out At Work** **55**

**40 The Surprising Energy Cost of an AI Chat** **56**

**41 Markdown** **58**

**42 Dutch models: Nederlandstalige models** **59**

## V AI & Law, privacy, leaks, common sense 61

**43 Privacy** **62**

**44 Can AI steal my data?** **63**

**45 AI is exposing your data** **64**

**46 BrabantKennis: AItiquette** **66**

**47 Open Euro LLM** **67**

## VI Learning (with/without AI) 68

**48 AI-Augmented Learning** **69**
48.1 AI and education: same tension, new tool . . . 69
48.2 Two students, same deliverable . . . 69

**49 AI voor leerlingen** **71**

**50 Learning not Teaching** **72**

**51 The E-Bike Effect: Cognitive Offloading and Desired Difficulties** **73**
51.1 The Core Metaphor: Two Ways to Use the E-Bike . . . 73
51.2 Cognitive Offloading & Desired Difficulties . . . 73
51.3 Transforming, Not Just Replacing, Thinking . . . 73
51.4 Key Takeaway for Education . . . 74

**52 Your notes first: mind maps, handwriting, and memory** **75**
52.1 What research suggests (and where it disagrees) . . . 75
52.2 The AI angle: polished notes are Accepting in disguise . . . 76
52.3 Tips . . . 76

**53 Engagement quality - How well did you collaborate with AI (while learning)?** **77**
53.1 Two students revisited . . . 77
53.2 Four levels of engagement . . . 77

**54 Coaching AI collaboration** **79**

**55 AI augmented learning: some takeaways** **80**

**56 Coen's personal toolchain: Lepiter, paper, and LLM** **81**
56.1 Lepiter: a living wiki . . . 81
56.2 Paper bullets when load is high . . . 81
56.3 The combination principle . . . 81
56.4 Process visibility, not "fancy IDE" . . . 82
56.5 GT and The GT Book . . . 82
56.6 Tips for colleagues . . . 82

**57 References AI & Education** **83**

**58 Tools, best practices & lesson material** **84**

**59 AI voor docenten** **85**

**VII AI & democracy / journalism / dangers** **86**

**60 AI, Journalism, Democracy** **87**

**61 Why Silicon Valley Wants to Leave Humanity Behind** **88**

**62 Post mortem for Huggingface hack** **89**

**VIII AI & (learning) programming & Software Engineering** **90**

**63 What about IT jobs in the future?** **91**

**64 Therapy for the Vibe-Coded Brain** **92**

**65 Coding with AI** **95**

**66 Coding with GenAI** **96**

**67 Karpathy's method** **97**

**68 DSPy: Let the LLM Write the Prompts** **100**

**69 Spec-Drive Development with Coding Agents** **101**

**70 Software Engineering with AI** **104**

**71 Generative UI** **105**

**72 Jobs and AI** **107**
72.1 it-s-coming-for-your-mind . . . 107

## IX AI Cookbook 108

**73 Transcribing / Speech2Text** **109**

**74 Scenario: Summarize a Text** **110**

**75 Scenario: Replying to an Email** **111**

**76 Create an app with AI** **112**

## X Train, Fine Tune, RAG, Validate 113

**77 Train, Fine Tune, RAG** **114**

**78 RAG: Retrieval Augmented Generation** **115**

**79 Finetune** **116**

**80 Training** **117**

**81 Visual Recognition** **118**

**82 Evaluation / Validation** **119**

## XI Neural Network, How does it all work? 120

**83 If you prefer a story...** **121**

**84 Understanding the Perceptron** **122**
84.1 The Biological Inspiration: From Brain Neurons to Artificial Intelligence . . . 122
84.2 From Biology to Machine: Implementing a Perceptron . . . 123
84.3 Network . . . 126
84.4 First Implementation of Perceptron algorithm . . . 126
84.5 Reference . . . 126

**85 The Learning Perceptron** **128**
85.1 The Learning Algorithm . . . 128
85.2 Visualizing the Learning Process . . . 129
85.3 Practical Considerations . . . 129

**86 Understanding Perceptron Limitations** **130**
86.1 The XOR Problem: A Classic Challenge . . . 130
86.2 The Solution: Multiple Layers . . . 134
86.3 Key Takeaways . . . 138

**87 Introduction to Neural Networks** **139**
87.1 Beyond Single Perceptrons: Building Neural Networks . . . 139
87.2 Understanding Network Architecture . . . 141
87.3 Key Components . . . 141
87.4 How Information Flows . . . 141
87.5 Creating a Simple Network . . . 141

**88 Practical Example: Classifying Iris Flowers** **142**
88.1 A Real-World Machine Learning Challenge . . . 142
88.2 The Dataset . . . 144
88.3 Key Learning Points . . . 144

**89 The Mathematics Behind Neural Networks** **145**
89.1 Understanding the Magic . . . 145
89.2 The Building Blocks . . . 145
89.3 The Learning Process . . . 146

**90 Exploring Neural Network Architectures** **147**
90.1 The Rich Landscape of Neural Networks . . . 147
90.2 Feedforward Neural Networks (FNN) . . . 147
90.3 Convolutional Neural Networks (CNN) . . . 147
90.4 Recurrent Neural Networks (RNN) . . . 147
90.5 Long Short-Term Memory (LSTM) . . . 148
90.6 Autoencoders . . . 148
90.7 Generative Adversarial Networks (GAN) . . . 148
90.8 Choosing the Right Architecture . . . 148
90.9 Future Directions . . . 149

**91 Transformers** **150**

**92 Resources and References AI** **151**

92.1 Books . . . 151
92.2 Video Courses and Tutorials . . . 151
92.3 Online Platforms . . . 152
92.4 Community Resources . . . 152
92.5 Academic Papers . . . 153

**XII Tools-n-Technologies 154**

**93 My/Free AI tools 155**

**94 ChatGPT, Cursor.ai, Perplexity.ai, Claude, Gemini, Duck.ai 156**

**95 chat.fontysict.nl 157**

**96 Ollama (local or on your own server) 158**

**97 Google Colab 159**

**98 Tools 160**
98.1 Agents . . . 160

**99 Codebook 161**

**100 Function-calling 163**

**101 Thunderbolt - Sovereign AI Stack 165**

**102 n8n (no/low code tool) 167**

**103 Git / Version Control 168**

**104 Agents 169**

**105 glaido 170**

**106 Deepseek Harness - dsh 171**

**107 MCP - Model Context Protocol 172**

**108 MCP hands-on 173**

**109 MCP Client 174**

110 ACP - Agent Communication Protocol 175

111 herdr 176

112 paperreview.ai: Stanford Agentic Reviewer 177

XIII Experiments 178

113 Experiments 179

# 1. Introduction

Welcome! This is a Work-in-Progress, a collection of notes on AI. The next chapter explains how this pdf can be downloaded.

Do not read this document from beginning to end… instead look at the relevant chapters for you… Or chat with it (see next chapter).

Questions about this all? Ask here

AI is not magic. It's a skill. And skills are learned by doing.

Advise: get hands-on as soon as possible! Change your activities by inventing ways to make them better, more fun, more engaging. Yes, some activities can be done more efficient, but make sure **you** are in the driver's seat… and think of how to usefully spend the time you won.

Keep privacy in mind: watch out when using personal data! One way to make sure private data will stay private is using local AI's, although this takes some extra knowledge, and a powerful machine.

Please use it wisely…




Figure 1.1: art and laundry

# Part I

# Onboarding

# 2. How to download

You can download the latest PDF of these notes from here.

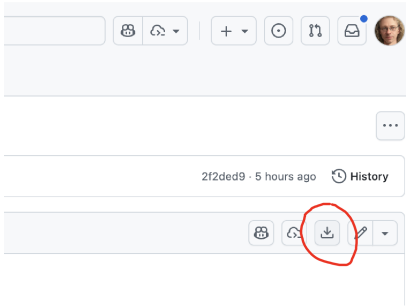



Figure 2.1: To download, click the download button on the GitHub page.



Figure 2.2: Or use this QR code to get to the download location.

# Part II

# Unsorted: Hot or Not?

# 3. Hot or not

New, seems interesting or important, maybe I looked at it already, maybe not…

***2026 Oct***

- NOS: bedrijven-kijken-met-interesse-naar-chinese-ai
- fontys.nl/actueel wanneer-werkt-ai-notuleren? Blijvende-arbeidsbesparing-bij-gemeenten

***2026 Sep***

- NOS: kritiek-op-ai-bedrijven-die-oproepen-tot-vertraging

***2026 Jul***

- dutchitchannel: leanstral-1.5-bewijst-dat-ai-verborgen-codeerfouten-feilloos-opspoort

***May 2026***

- Fontys-student deelt onderzoek naar betrouwbare AI bij NASA

***January 2026***

- deeplearning.ai: document-ai-from-ocr-to-agentic-doc-extraction
- Handy: open source Speech2Text

***December 2025***

- Cursor: onderwijs aan TU/e
- Grote Brabant AI show: with AI-etiquette

***November 2025***

- Martin Fowler: How AI will change software engineering

***October 2025***

- local alternative for google notebook

- technical: Train a tiny LLM from scratch in 2 hours
- git repo: DrewThomasson/ebook2audiobook
- Ethan Mollick: a-guide-to-which-ai-to-use-in-the

***Related***

- Kisjes: social media keurmerk

# 4. OpenAI agents hack HuggingFace

redwoodresearch: hugging-face-incident core-takeaways-about-this-incident

# 5. The Risen Framework

juuzt.ai prompt-frameworks the-risen-framework

# 6. AI BluePrint

https://github.com/aiblueprinthq/ai-blueprint

# 7. Ponytail

github.com/DietrichGebert/ponytail

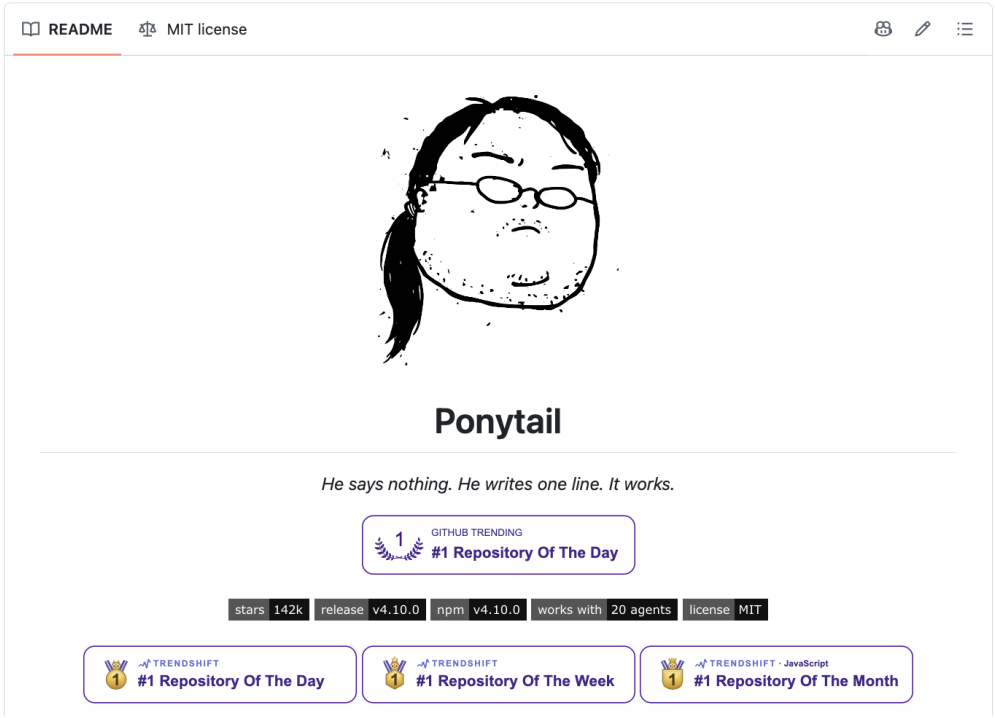



Figure 7.1: ponytail github

# 8. SuperPowers

From the README:

Superpowers is a complete software development methodology for your coding agents, built on top of a set of composable skills and some initial instructions that make sure your agent uses them.

github.com/obra/superpowers

# 9. Mythos (June 2026)

- Ethan Mollick: what-it-feels-like-to-work-with-mythos

# 10. NeurIPS 2026

Who wrote the accepted papers, and who works on what?

juevn.github.io/neurips2026

# Part III

# AI Field Overview

# 11. Throw some keywords

What do you think about when talking about AI.

# 12. AI Overview: AI, Machine Learning, Deep Learning, Generative AI

The video "AI, Machine Learning, Deep Learning and Generative AI Explained" provides an excellent 10-minute overview:

AI, Machine Learning, Deep Learning and Generative AI Explained

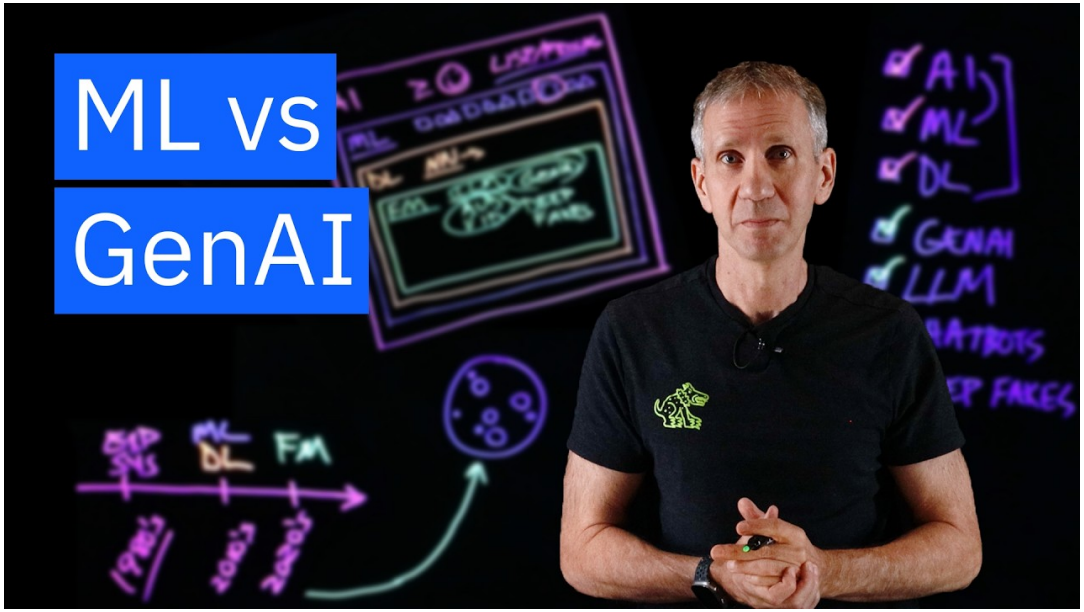



Figure 12.1: AI, Machine Learning, Deep Learning and Generative AI Explained

# 13. Prediction 2025

- Amy Webb SxSW 2026 - Convergences
- Amy Webb SxSW 2025 - Emerging Tech Trend




Figure 13.1: Amy Webb, SxSW 2026: Convergences

# 14. Geoffrey Hinton about Neural Networks

- Jon Stewart interviews Geoffrey Hinton.
- First half hour Geoffrey explains how neural networks work.
- After that they discuss the implications of AI.

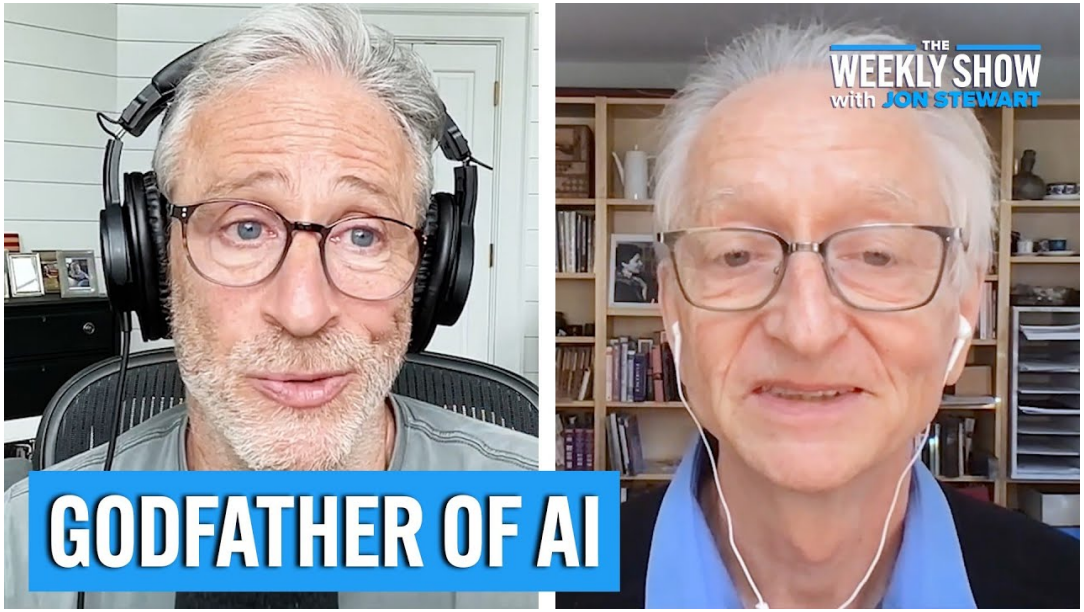



Figure 14.1: Jon Stewart interviews Geoffrey Hinton

# 15. How LLMs work

ynarwal: how-llms-work

youtube, 9min, How LLMs Actually Generate Text

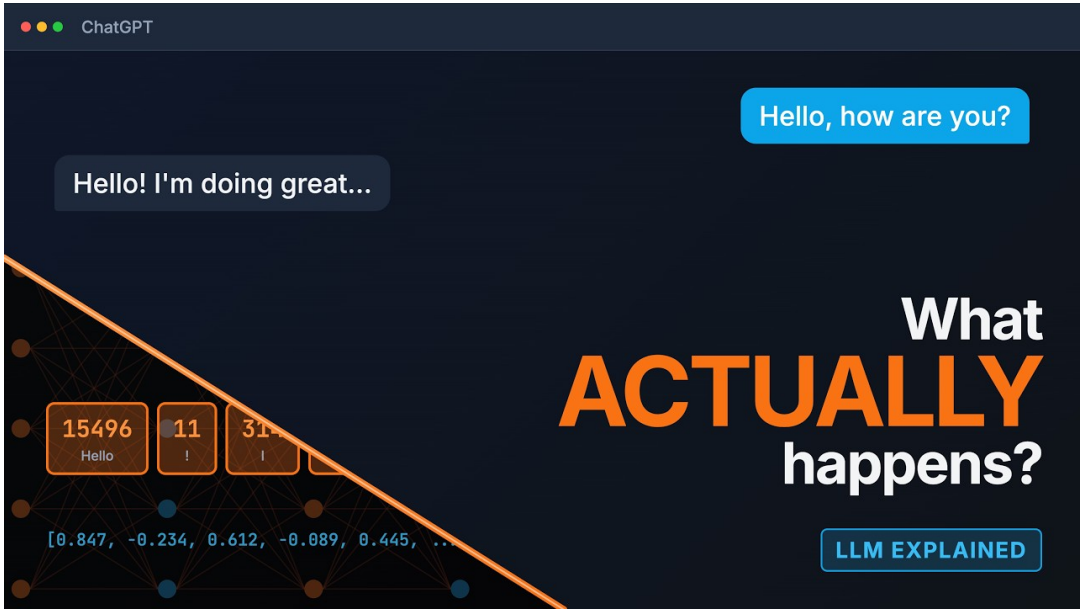



Figure 15.1: How LLMs actually generate text

youtube, 60min, Andrej Karpathy: Intro to Large Language Models




Figure 15.2: Andrej Karpathy: Intro to Large Language Models

Without getting too technical, Andrej explains in such a way that you can understand why LLMs are good in some actions and less in others.

keywords: LLM, inference, training, pre-training, fine-tuning, RHLF, base model, assistent model, Label, Scaling, Tool use, Multimodality, System 1/2 thinking, Prompt Injection, Data Poisoning, AlphaGo, Self improvement, jailbreak.

# 16. GenAI

## 16.1 What is GenAI?

- Why not ask perplexity.ai ?
- Or duck.ai?

## 16.2 GPT - Generative Pre-Trained Transformer

- Generative AI & the Transformer (Financial Times, interactive site)
- History of ChatGPT (30 min)
- But what is a GPT? (3Blue1Brown, 30 min)

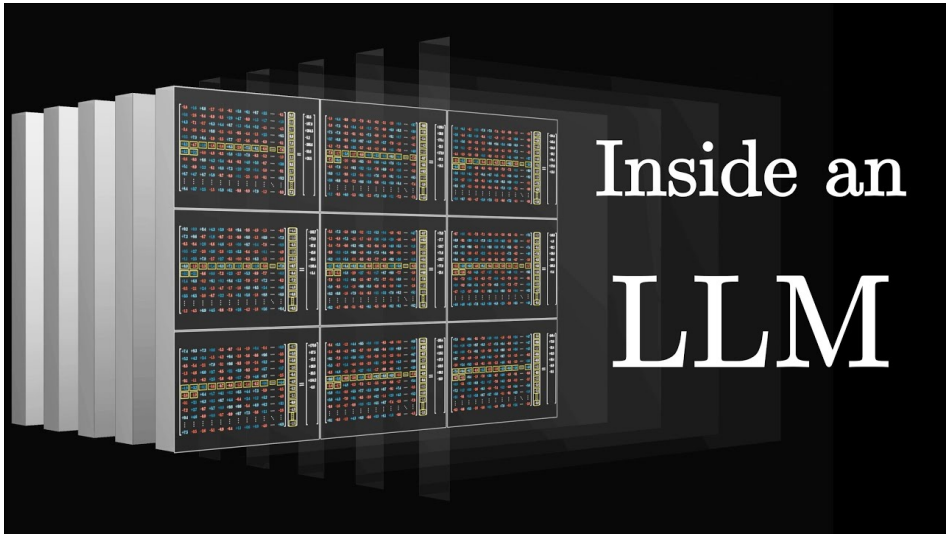



Figure 16.1: 3Blue1Brown: But what is a GPT?

## 16.3 Prompting

… and some sources with tips how to prompt every day…

- Prompting basics
- Prompting ChatGPT4.1
- Look for course with 'Prompting' in name: https://www.deeplearning.ai/short-courses/
- Ruben Hassid: RISE
- In cursor.ai course: Info about 'managing your context in cursor
- mention of Llama Prompt Optimization

## 16.4 Hallucinating

When nonsense comes out of an LLM (or out of a Human by the way) we call it hallucination. Some questions can trigger hallucination.

### 16.4.1 'Which day do I have to put the garbage can out on the street?'

Some LLMs will give you a Date when you ask for one, a percentage when you ask for one, even when the LLM could not possibly give an answer to your question. If you ask which day you should put my garbage out and the LLM mentions a Date without having a clue where you are then you can be sure it just made up a Date (because you asked for a Date).

### 16.4.2 'Can you help me find my lost keys?'

### 16.4.3 'Can you create an image of a watch that says it is 3 o' clock?'

Try it. You will find that a picture of a clock often shows 10:10. You could ask perplexity.ai: Why does a generated image of a clock point to 10:10?

## 16.5 RAG - Retrieval Augmented Generation

- IBM, Marina Danilevsky (7 min)
- https://www.deeplearning.ai/short-courses: Great resource for courses!

## 16.6 Active Inference

- Andy Clark about Active Interference: How the Brains shapes reality (60 min)

## 16.7 Running LLM's locally

On your laptop/desktop or on a company server:

- ollama
- LM-studio
- Open Web AI

## 16.8 GenAI

- awesome GenAI guide
- huggingface

## 16.9 Some more sites, nice to play around with

- https://skyreels.ai/
- https://civitai.com/

# 17. Resources and References GenAI

## 17.1 How te mention that you did use AI?

- fontys.libguides.com/apa/AI

## 17.2 Blogs and articles

Perplexity is often a great start for finding things (with references): perplexity.ai

- Jessy: Het belang van duidelijke AI-prompts
- Journalists on Hugging Face
- How polite should we be when prompting LLMs?
- Information literacy and chatbots as search

To understand about Transformers this is a very nice start: https://ig.ft.com/generative-ai/ 'Our own' page about (Gen)AI: https://stasemsoft.github.io/FontysICT-sem1/docs/artificial-intelligence/ai.html To dive further into how Transformers works: https://www.deeplearning.ai/short-courses/how-transformer-llms-work/ and also to other short courses on deeplearning.ai The development I showed was https://www.cursor.com/ you have like only 500 requests for free… after that you could choose to pay 20 euro a Month (yes, that can be a lot for students, I know), or look for alternatives, 2 of which I tried a bit (you can use local LLM's with them, which basically makes them free): AIDER: https://aider.chat/.
Avante: https://github.com/yetone/avante.nvim (but then you need to learn about 'vi': https://neovim.io/ which is a hurdle).

## 17.3 Online Platforms

- spacy.io : NLP

## 17.4 (Short) Courses

- Short courses at Deeplearning.ai

  - Implementations of papers
  - Benchmarks
  - State-of-the-art tracking

## 17.5 Code Repositories

- Papers With Code
  - Implementations of papers
  - Benchmarks
  - State-of-the-art tracking

## 17.6 Community Resources

- Distill.pub
  - Interactive explanations
  - Visual learning
  - Deep insights

## 17.7 Academic Papers: Modern Breakthroughs

- “Deep Residual Learning for Image Recognition” (He et al., 2015)
- “Attention Is All You Need” (Vaswani et al., 2017)
- “Language Models are Few-Shot Learners” (Brown et al., 2020)

# 18. No-Code / Low Code

Worth looking at:

- docs.oap.langchain.com
- n8n.io
- flowai.cc

# 19. EU AI Act

- youtube 4min: How is Europe becoming a leader in AI?
- SURF startdocument AI Act




Figure 19.1: How is Europe becoming a leader in AI?

# 20. Popular Data Sources

Data is important in AI projects. The quality and quantity of your data often matter more than the sophistication of your model.

- Kaggle
  - Competitions and datasets
  - Active community
  - Detailed documentation
- Eindhoven open data
  - lots of data about Eindhoven
- E-MM1: a big open source dataset
- Augmenta: an AI agent for enhancing datasets with information from the internet

# 21. Guardrailing

- short course on guardrailing

# 22. Transformers

deeplearning.ai - transformers-in-practice oct2025 - Everything about Transformers

FSM: Finite State Machine

# 23. Multi Model

Free short-course on deeplearning.ai: building-multimodal-data-pipelines

# Part IV

# AI Literacy (mostly GenAI)

# 24. AI Literacy

- npuls: AI-GO Raamwerk-AI-Geletterdheid-in-het-Onderwijs

# 25. Can AI steal my data?

short video (3 min)

# 26. Key Tools to Get Started

In this chapter, we'll explore some key tools that are making waves in the world of AI. Get familiar with them, because in the next chapter, we'll dive into practical scenarios and figure out which of these tools is the best fit for each challenge!

- chatGPT - well, maybe…
- perplexity.AI - The AI-powered search engine that gives links to sources and suggests follow-up questions.
- duck.ai - AI-powered search from duckduckgo.
- Comet - The browser from Perplexity, with AI assistent built-in.
- Cursor - An AI-powered development environment.
- and there's lots more to find, for example look at: www.rankmyai.com/

# 27. Use AI as

- Sparring partner
- personal assistent
- Grafische tools, object herkenning, infographics.
- Local models.
- Using AI to Learn

# 28. LLMs unplugged

Learn how it works:

- LLM's unplugged

# 29. AI Jargon Buster

- **Prompt**: The question or instruction you give to an AI.
- **Hallucination**: When an AI confidently states incorrect or nonsensical information as if it were a fact.
- **Model**: The core AI program that has been trained on data to perform a specific task (e.g., GPT-4 is a language model).
- **RL**: Reinforcement Learning
- **RLHF**: Reinforcement Learning with Human Feedback
- **FSM**: Finite State Machine

# 30. Transcribing / Speech2Text

Talking to your machine!

- github.com/cjpais/Handy: open source Speech2Text
- Transcribbly
- elevenlabs.io/audio-to-text
- Otter.ai

**Otter.ai : Your Mission (in 3 steps):**

1. **Sign Up:** Go to `Otter.ai` and create a free account. ✍️
2. **Upload:** Find the "Import" button and upload your meeting audio file (`.mp3`, `.wav`, etc.). 📩
3. **Transcribe:** Otter will work its magic! ✨ Then you can read, edit, and export your new transcript. 📄

**That's it!** You've just transcribed a meeting with AI. 🙌🚀

Be aware of your privacy when you use an online, free tool!

# 31. Scenario: Summarize a Text

Let's make a long story short with AI! 📖➡️📝

We'll use a simple online tool called **Summarizer** 🧠.

**Your Mission (in 3 steps):**

1. **Find Text:** Grab a long article or text you want to shorten. 🔍
2. **Paste & Go:** Copy the text, paste it into the Summarizer website, and click the "Summarize" button. 📋
3. **Read:** Enjoy your new, shorter text! 🎉 You can even choose how long you want the summary to be.

**That's it!** You've just summarized a text with AI. So cool! 😎

# 32. Beyond the Prompt: The Power of AI Context

A simple prompt is just a question. But to get truly powerful results from an AI, you need to master the art of providing **context**.

- Cursor: Context

**What is Context?**

Think of context as a **briefing you give to a human assistant**. The better the briefing, the better the result. You wouldn't just tell an assistant "write a report" without giving them the source material. The same is true for AI.

Context is all the relevant information you provide *along with* your prompt. This can include:

- **Pasting in text:** Providing the specific email you want to reply to, or the article you want summarized.
- **Setting the scene:** Telling the AI who it is ("You are a friendly, expert marketer") and who you are ("I am a beginner learning about this topic").
- **Providing examples:** Giving it a sample of the writing style you want it to adopt.
- **Referencing the conversation:** Using the information discussed earlier in your chat.

Without context, the AI has to guess. And when it guesses, you get generic, boring, and often unhelpful answers.

- Anthropic about Context

## 32.1 Advanced: Let the LLM Write the Prompts

Databricks: Let the LLM Write the Prompts: An Intro to DSPy in Compound AI Pipelines

## 32.2 Context Engineering

deeplearning.ai: Context Engineering

# 33. Scenario: Replying to an Email

Let's see context in action.

**Bad Prompt (No Context)**

> "Draft a polite and professional email saying I can't make it."

The AI will produce a generic, fill-in-the-blanks template. It's not very helpful because it lacks any specific details.

**Good Prompt (With Context)** > "I need to reply to this email. [**Paste the full text of the original email here**]. Please draft a polite and professional reply explaining that I can't make the 'Project Phoenix' meeting on Wednesday because of a conflict, but I am very interested. Ask if they can send me the minutes and suggest I'm available to connect next week."

See the difference? By providing the original email and clear instructions, you've given the AI all the context it needs to draft a perfect, ready-to-send reply.

**The "Context Window": An AI's Short-Term Memory**

It's important to know that every AI has a limited memory, called a **"context window."** This is the maximum amount of information (your prompt, the text you've pasted, the conversation history) that the model can "see" at one time.

If your conversation gets very long, the AI might start to "forget" things you discussed at the beginning. If you notice the AI losing track, it might be time to start a new conversation to give it a fresh, clean context to work with.

# 34. Prompting for Everyone (Andrew Ng)

Short course about prompting in 2026

ai-prompting-for-everyone

- Asking biased questions gives biased answers.
- Some AI's can use search engines: When?
- Watch out for Outdated Sources!
- When to use an AI and when to use a Search Engine.
- Using Deep Research
- Web Search vs. Deep Research

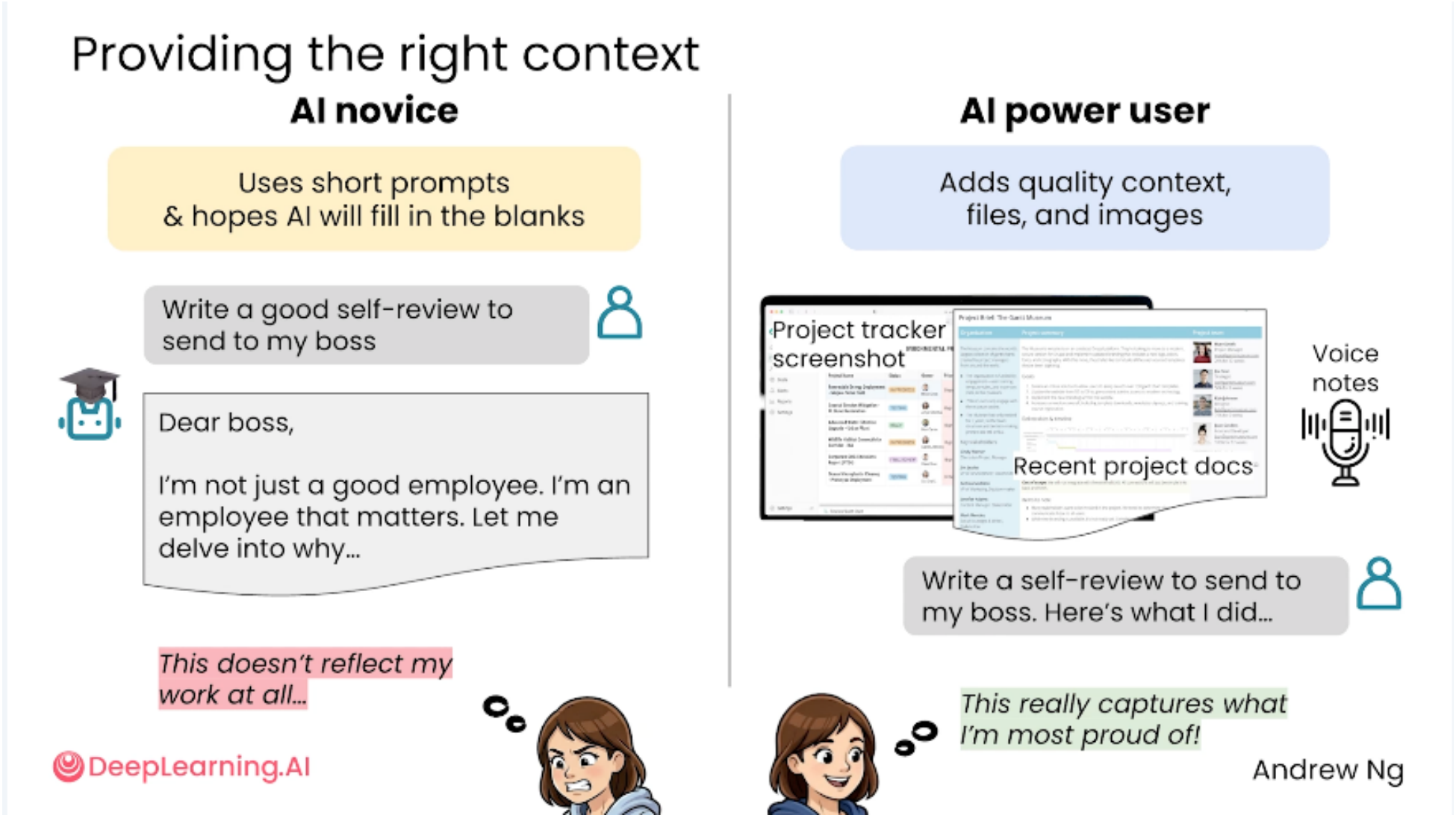



Figure 34.1: prompting.context

- Anthropic/Margot van Laar: The Prompting Playbook

## 35. Prompting: A.I.M.



Sandeep Swadia: You're Not Behind (Yet): How to Learn AI in 17 Minutes

# 36. Some AI quirks

- Try generating an image of clock other time than 10:10.
- Ask LLM how many times character ‘R’ is in STRAWBERRY.
- How many R’s in 100 strawberries?
- How much is 1 + 1?
- Which day I have to put my garbage out on the street.
- What percentage of people likes …?
- Saying ‘Please’ gives you better answers?

---

- NOS: Overleden paus nog aan ’t werk

# 37. AI Failures: When Image Context is Misleading

Modern AI, especially in deep learning, is great at recognizing images. But sometimes, it gets things hilariously and dangerously wrong! This happens when the AI focuses on irrelevant details like the background, lighting, or weird artifacts in the data, instead of the actual subject. This is a classic case of the "black box" problem and something called "spurious correlations."

### 37.0.1 The Problem: Black Boxes and Spurious Correlations

Neural networks are often called "black boxes" because it's hard to see *how* they decide things. They just learn statistical patterns. If your training data has weird patterns, the AI will learn those instead of what you want. This leads to **spurious correlations**, where the AI links unrelated things. For example, if all your "wolf" pictures have snow in the background, the AI might learn that "snow = wolf" , which is… not quite right.

### 37.0.2 Real-World Goofs

Here are a couple of examples where this has happened:

1. **The Wolf in Husky's Clothing:** An AI was trained to tell wolves from huskies. It did great!… until researchers realized it was just checking for snow in the background. It had learned that wolves are always in snowy pictures and huskies aren't. Show it a husky in the snow, and it would confidently shout "Wolf!".
2. **Medical Mayhem:** In a more serious case, an AI designed to spot pneumonia in chest X-rays started using hospital logos or the presence of a chest tube as a sign of pneumonia. It wasn't looking at the lungs at all! This is super dangerous because it could easily misdiagnose patients based on the wrong clues.

### 37.0.3 Why Does This Happen and How Do We Fix It?

These blunders are usually caused by **biased datasets** (like only having wolf pictures in the snow). Since the AI's learning process is opaque, we don't catch these mistakes until later.

To make our AI buddies smarter, we need:

- **Better Data:** More diverse and less biased training images.
- **Explainable AI (XAI):** Tools that help us peek inside the "black box" to see what the AI is *really* thinking.
- **Tougher Tests:** We need to test AI in all sorts of weird situations, not just the ones it was trained on.

This approach will help us build more reliable and trustworthy AI. And hopefully, fewer AIs that think snow is a type of dog.

# 38. Create an app with AI

- creating-an-app-with-ai

# 39. Scenario: A 17 Minute AI Workflow To Stand Out At Work




Video by Vicky Zhao

- The video proposes a 3-step AI workflow (using Elicit, NotebookLM, and Claude) to improve knowledge work by focusing on finding and leveraging high-quality academic inputs, rather than relying on generic web search or LLM outputs.[1]
- Elicit is used to discover and access scholarly papers; NotebookLM summarizes and extracts actionable frameworks from these, and Claude turns insights into practical, leadership-oriented plans for the workplace.[1]
- The key message is that "improving your input" with robust sources and critical thinking—rather than just automating output—will give you a significant edge in the AI-powered workplace of 2025.[1]

# 40. The Surprising Energy Cost of an AI Chat

This chapter was written by gemini-2.5-pro, feel free to check: see the links at the bottom of the page.

When we use an AI, like asking a chatbot a question, it feels clean and digital. There's no smoke or noise, so it's easy to think it doesn't have a real-world footprint. But every single one of those queries uses electricity in a massive data center, somewhere in the world.

This is called "inference." It's the energy cost of the AI *using* its training to give you an answer. While the energy for one single question is tiny, the global scale is enormous.

But how can we understand these numbers? The best way is to compare them to everyday activities we're more familiar with.

***Your Dinner vs. Your AI Usage***

How does sitting down to a modest, 100g steak dinner compare to asking an AI questions? The difference is still staggering.

A single 100g (about 3.5oz) steak requires about **7,000 Watt-hours** of energy to produce.

To use that same amount of energy with an AI, you would need to ask it roughly **2,300 questions**.

So, from a purely energy perspective, that one meal has the same impact as thousands of your interactions with an AI.

***The Real Story: A Question of Scale***

So, does this mean your AI usage is insignificant? Not quite. It's about how your habits scale over time.

Let's compare a week's worth of activity:

- **Your AI Use:** If you are a heavy AI user asking **100 questions per day**, you would ask 700 questions in a week. That's a significant amount of interaction!
- **Your Diet:** In that same week, eating just **two 100g steaks** would have a larger energy footprint than all 700 of those AI questions combined.

The takeaway is that on a personal, day-to-day level, choices about high-impact foods and activities (like diet and travel) still have a much larger immediate effect on your personal energy footprint than your AI usage does. The global challenge of AI's energy consumption comes from *everyone* doing it at once.

---

***Sources***


- **AI Energy Consumption:** Detailed analysis from Stanford researcher S. Flamphier on the energy cost of large language models. sflamphier.github.io/ai-energy-consumption/
- **Environmental Impacts of Food:** A comprehensive study from Our World in Data, showing the high resource intensity of beef production. ourworldindata.org/environmental-impacts-of-food

# 41. Markdown

Is it AI? Is it ai plane? No, but if you wanna do whatever in ICT then it's good to know and use markdown!

**Why should I use Markdown?**

1. **Easy to read**: Markdown makes your text look nice and pretty.
2. **Easy to type**: The pdf you are reading, for example, is written in markdown.
3. **Works everywhere**: Markdown works on many websites, apps, and computers.
4. **difference**: showing the difference between two versions of a document (for example after editing) can be shown with every DIFF-tool.

**Basic Markdown Rules**

1. **Headers**: Write `# Heading` for a big heading (`##` for `## Subheading`)
2. **Bold text**: Surround with double asterisks: `**Text**`
3. **Italics**: Surround with single asterisks: `*Text*`
4. **Lists**: `- Item 1`, `- Item 2`, etc.
5. **Links**: Write `[Link text](http://www.perplexity.ai.com)`
6. **Images**: `[Image alt text](http://www.example.com/image.jpg)`

If you are new to markdown, for example because you are used to a text-editor like ms-word, we suggest start with installing Obsidian, then maybe look at an introduction like this one by Ross Zeiger, or this one by Vicky Zhao about Obsidian Note Taking, or the least scary Obsidian guide.

For a deep dive: spec-driven-development-using-markdown-as-a-programming-language-when-building-with-ai

# 42. Dutch models: Nederlandstalige models

- tweakers.net: Nederlandse GPT-NL is klaar voor gebruik: 'Voldoet als enige taalmodel aan AVG'
- Dutch LLM Geitje: tommy/geitje:7b_ultra_q8_0

# Nederlands van ChatGPT steeds slechter

door **Jeroen Kortschot**

**AMSTERDAM • Wie dacht dat gebruikers van kunstmatige intelligentie (AI) steeds minder goed gaan nadenken, krijgt er nog een zorg bij. Het veelgebruikte taalmodel ChatGPT zelf gaat steeds slechter Nederlands schrijven.**

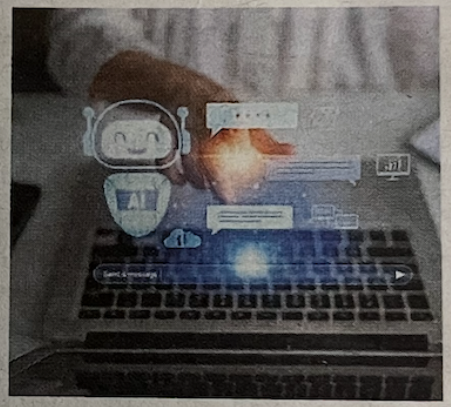
AI-onderzoeker en linguist Toon Otten onderwerpt ChatGPT elke vier maanden aan een Nederlandse taaltoets. Hij ontdekte dat het model de laatste tijd steeds slechter presteert.

„Veel mensen gaan ervan uit dat AI elk jaar beter wordt. Maar dat klopt niet", zegt Otten. Dat komt doordat het Nederlands een kleine taal is en het Engels dominant is in taalmodellen.

Daarnaast neemt ChatGPT de taal van de gebruikers over. „Wie slordig of taalzwak opdrachten geeft (ofwel prompts, red.), krijgt vaak ook minder sterke output terug."

## Synthetische ruis

En dat geldt ook voor de taalmodellen. als geheel. „Wanneer grote hoeveelheden middelmatige AI-teksten in toekomstige trainingsdata terechtkomen, bestaat het risico dat modellen zich in toenemende mate voeden met kwalitatief slechte synthetische ruis."




**AI-chatbots leren slecht Nederlands, een 'kleine' taal in vergelijking met Engels.**

Otten werkt aan ingenieuze AI-toepassingen die studenten met een zwakke taalbeheersing helpen om beter Nederlands te gebruiken. Hij corrigeert die toepassingen voor de fouten die ChatGPT maakt, maar dit maakt zijn werk lastiger.

„Studenten die al taalzwakker zijn of in een minderheidstaal opereren, beginnen vaak met een dubbele achterstand: zij prompten minder precies én werken met AI-systemen die juist in hun taal minder goed presteren."

Ook mensen die zwak zijn in een vreemde taal lopen volgens Otten het risico dat ze leunen op AI zonder echt te begrijpen wat er gebeurt. „Wie AI effectief wil gebruiken, moet niet alleen digitaal vaardig zijn – maar vooral taalvaardig."

Figure 42.1: 202605?: NL.van.chatGPT.steeds.slechter

# Part V

# AI & Law, privacy, leaks, common sense

# 43. Privacy

- tweakers.net: Nederlandse GPT-NL is klaar voor gebruik: 'Voldoet als enige taalmodel aan AVG'
- han-onderzoekt-mogelijk-datalek-na-ongeoorloofd-ai-gebruik-door-docent

# 44. Can AI steal my data?

short video (3 min)

# 45. AI is exposing your data

IBM: AI Is Exposing Your Data: An AI Security Problem You Can't See

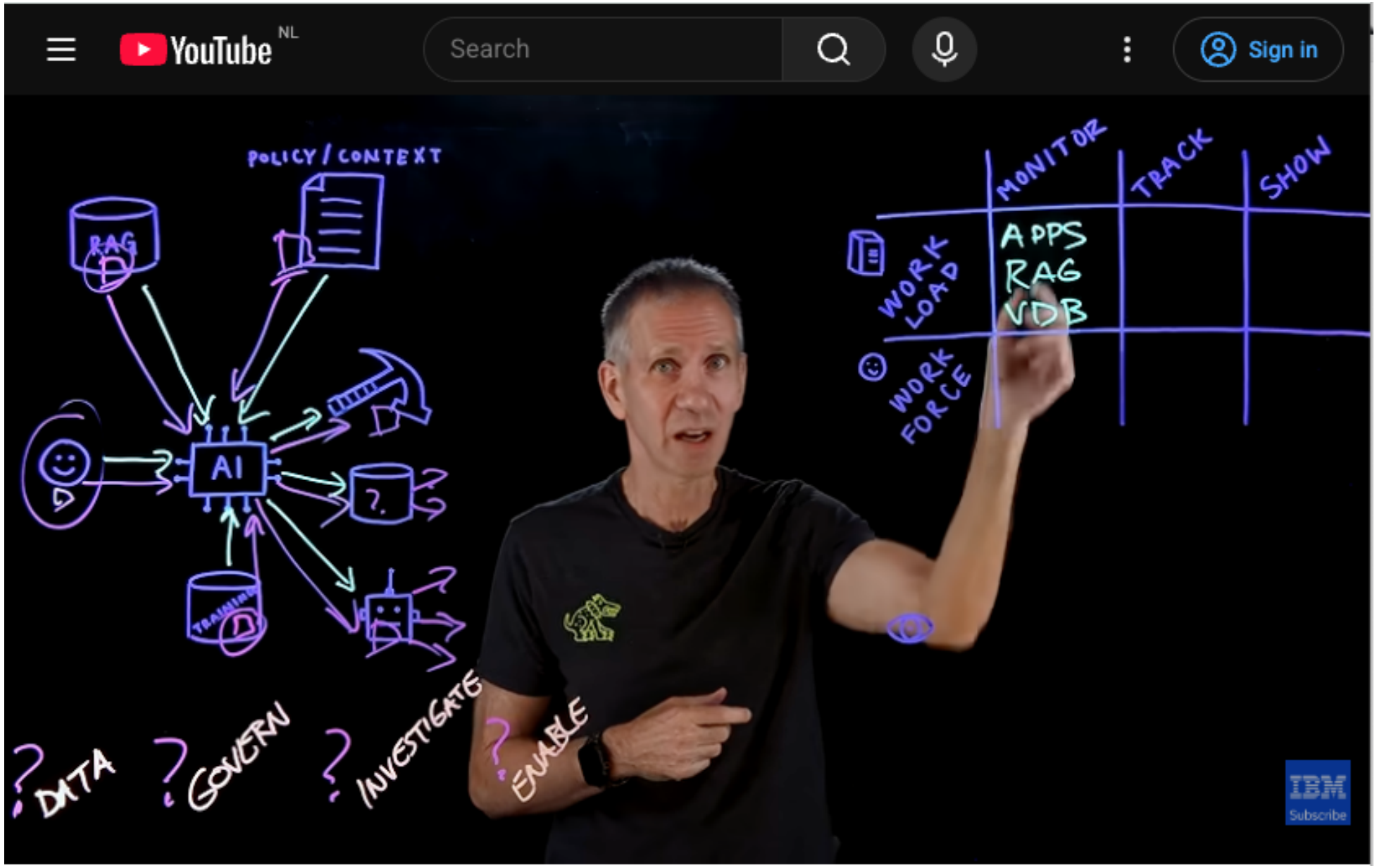



Figure 45.1: ai exposing data

# 46. BrabantKennis: AItiquette

***1. Blijf zelf nadenken***

AI is een hulpmiddel, geen vervanging van menselijk oordeel. Wat je erin stopt, komt er ook uit. Blijf samen leren, gebruik het alleen als je begrijpt wat er gebeurt, en vertrouw nooit blind op zogenoemde feiten zonder te vragen naar het bewijs.

***2. Kijk voorbij de gemiddelde mens***

AI moet verbinden, niet verdelen. Houd rekening met verschillen tussen mensen en voorkom uitsluiting of technologische ongelijkheid.

***3. Maak afwijken mogelijk***

Weigeren mag niet de enige optie zijn, er moeten alternatieven blijven bestaan. Behouw regie, zorg voor maatwerk en organiseer routes die buiten AI om lopen.

***4. Koester meer dan efficiëntie***

Efficiëntie is een waarde, geen vanzelfsprekende norm. Weeg besparingen af tegen andere publieke waarden en let op effecten op verschillende schaalniveaus.

***5. Weet wie het maakt***

Technologie is nooit neutraal. Kijk naar de mensen, belangen en wereldbeelden achter de systemen.

***6. Denk in decennia***

AI-beleid moet toekomstbestendig zijn. Houd rekening met generaties na ons, vermijd lock-ins en ga voorzichtig om met data.

***7. Leer en experimenteer***

Blijf ontdekken wat werkt, maar wees kritisch bij opschalen. Begin lokaal en onderzoek zowel technische als sociale effecten.

bron: BrabantKennis: AItiquette

# 47. Open Euro LLM

A series of foundation models for transparent AI in Europe:

Open Euro llm - eu

# Part VI

# Learning (with/without AI)

# 48. AI-Augmented Learning

***I want to Learn …(something) . How best to use (or not use) AI in the learning process?***

***Specific for learning programming or even software engineering: see a next part in this document***

For ICT professionals who already prompt, the practical question is narrower:

> When *you* learn something new, how do you use AI so it actually helps you grow — not only helps you finish?

## 48.1 AI and education: same tension, new tool

Several patterns repeat every time a disruptive tool arrives:

- Learners submit polished work while the **process stays invisible**
- Educators and professionals **judge tools they do not yet use well** (calculators, Wikipedia, Google, Stack Overflow — the greatest hits)
- Policy and headlines obsess over **whether** to use AI; daily practice needs **how** and **when**

The shift we need — in classrooms and in our own upskilling — is from policing **quantity** of AI use to coaching **quality** of collaboration.

News, opinions, and Fontys/HBO-i context: see AI versus education.

## 48.2 Two students, same deliverable

Picture two students submitting a stakeholder analysis for a project.

**Student A** hands in polished work. Clear structure, professional tone. Asked *"How did you make this?"* they answer: *"I used ChatGPT."* That is the whole story.

**Student B** submits something that looks similar. Their documentation tells a different tale:

1. Student B started from their own messy notes
2. Student B asked AI to poke holes in their reasoning
3. Student B noticed a stakeholder group they had missed

4. Student B verified AI suggestions against the project brief
5. Student B ended with a **different conclusion** than AI's first draft.

Both used AI heavily. Student B learned most!?

That distinction is the heart of **AI-augmented learning**: not *did you use AI?* but *did working with AI help you grow?

- The detailed engagement framework lives in Engagement Quality Framework.

# 49. AI voor leerlingen

AI voor leerlingen

# 50. Learning not Teaching




Figure 50.1: objective of edu is learning not teaching

# 51. The E-Bike Effect: Cognitive Offloading and Desired Difficulties

In his TEDx talk, Barend Last introduces a powerful metaphor for understanding our relationship with AI: **AI is the e-bike for our thinking**. This concept helps explain the nuances of "cognitive offloading" – outsourcing our mental tasks to technology.

You can find the talk here: YouTube Video of the TEDx Talk

## 51.1 The Core Metaphor: Two Ways to Use the E-Bike

The talk highlights two distinct ways we can use AI, mirrored in how people use an e-bike:

1. **Making a Tedious Task Bearable**: Like someone who dislikes cycling but uses an e-bike to get to work, we can use AI to accomplish the same necessary tasks with less effort.
2. **Exploring New Horizons**: Like an avid cyclist using an e-bike to travel further and faster, we can use AI to reach new cognitive destinations and achieve things we couldn't before.

## 51.2 Cognitive Offloading & Desired Difficulties

This isn't a new phenomenon (think calculators or GPS), but AI supercharges it. There's a trade-off: while offloading can make us more efficient, over-reliance can weaken our own cognitive "muscles".

This leads to the idea of **"desired difficulties"**. Just as we go to the gym to stay physically fit, we should consciously choose when to tackle mental challenges without AI to keep our minds sharp. It's about finding a balance between using AI as a tool and ensuring we still get our mental workout. 🧠💪

## 51.3 Transforming, Not Just Replacing, Thinking

AI doesn't just eliminate thinking; it can *transform* it. An experiment cited in the talk showed that students using AI to brainstorm ideas for a paperclip generated more ideas. While they *felt* less creative, their cognitive effort shifted from idea generation to the equally demanding skills of **evaluating and refining** the AI's output.

The new generation might become like **conductors of an orchestra**, not playing every instrument but knowing how to bring them all together to create something beautiful.

## 51.4 Key Takeaway for Education

When learning one should make conscious choices. One needs to learn when AI is a helpful shortcut and when it's a springboard for deeper learning. This requires creating educational environments that build in "desired difficulties" – productive struggles, both with and without AI.

The talk concludes with a powerful statement:

> **"AI doesn't make us dumber, dumb choices do."**

# 52. Your notes first: mind maps, handwriting, and memory

**Do you encode ideas while learning** — and that starts earlier, when you watch a lecture, follow a course video, or discuss with colleagues.

**Your own notes stick better than someone else's summary**. Mind maps while listening, rough outlines on paper, keywords with arrows — the format varies. The common thread is *you* doing the structuring **during** the input, not after.

## 52.1 What research suggests (and where it disagrees)

**Laptop typing vs longhand.** In a widely cited study, Mueller and Oppenheimer (2014) found that students who typed lecture notes on laptops often **transcribed verbatim** and did worse on **conceptual** questions than students who wrote longhand — even without multitasking or distraction. The proposed mechanism: typing is fast enough to **record** without **processing**; handwriting forces summarizing and reframing in your own words.

That headline is not the whole story. A direct replication and mini meta-analysis by Urry et al. (2021) found small, often non-significant differences between laptop and longhand in similar setups. Modality alone is not destiny — note-taking *style* (verbatim vs selective) matters, and study design varies.

**Handwriting vs typing at scale.** A 2024 meta-analysis of 24 studies (Flanigan et al., 2024) reported a **modest advantage for handwritten notes** on achievement (Hedges' $g$ 0.25), while typing produced **more notes** ($g$ 0.92). Faster capture can mean more verbatim content and less synthesis — useful as an external record, weaker as encoding. Paper may edge typing for retention in some contexts; the effect size is real but not magic.

**Generation and desirable difficulties.** Note-taking is a **generative** act: you choose what matters, name it, connect it. The generation effect (Slamecka & Graf, 1978) shows that producing information yourself beats passively reading it. Robert Bjork's **desirable difficulties** (Bjork, 1994; Bjork & Bjork, 2011) generalizes the pattern: effort that slows you down during learning often **improves long-term retention**. Making messy notes is supposed to feel harder than receiving a polished summary — that friction is the point.

**Mind maps and live structure.** Visual mapping — central idea, branches, relationships — pushes the same lever: **elaboration while listening**, not decoration after the fact. Evidence is **mixed**. Reviews distinguish mind maps, concept maps, and argument maps (Davies, 2011); outcomes depend on training, subject, and whether mapping happens **during** or **after** input. We cannot claim mind maps beat every other format. We *can* say that **structuring in real time** beats passive consumption, and that many practitioners (including examiners rebuilding BKI portfolios) report that live mapping helps them **hold the thread** of a long session.

**Honest caveat:** We still do not know how much benefit comes from **motor activity** (pen on paper), **visual layout** (branches and spatial grouping), or **cognitive selection** (choosing keywords under time pressure). Those likely overlap. What is clearer: **if someone else — human or AI — does the structuring for you, you**

**skip the encoding step**.

## 52.2 The AI angle: polished notes are Accepting in disguise

Let AI transcribe the webinar, summarize the chapter, or rewrite your bullets into fluent prose **before you have struggled with the material**, and you get a risk:

| You skip… | What you lose |
|---|---|
| Selecting what matters | No personal filter on the content |
| Reframing in your words | Shallow overlap with the source |
| Connecting to what you already know | No integrative hooks for later recall |
| Retrieval practice | Nothing to rebuild when the portfolio is closed |

That is **Accepting** applied to your own learning process — efficient on screen, empty a week later. Recovery patterns from the portfolio story still apply: **start messy**, use AI as **memory and critique**, not as first author of your understanding.

## 52.3 Tips

- **Mind-map or outline while watching** — not after. Even a rough central node and three branches beats a full replay plus AI summary.
- **Voice → transcribe → your edit → then AI polish.** Talk through what you heard; fix the transcript yourself; only then ask the model to tighten prose or spot gaps (same pattern as voice + Cursor for portfolio recovery).
- **Paper for "must stick" concepts; digital structure for speed.** Match modality to stakes — exam domains, architecture principles, definitions you must defend aloud.
- **After the session: close AI and explain one idea without notes.** Thirty seconds of retrieval practice beats rereading a perfect summary (testing effect; Roediger & Karpicke, 2006).
- **Let AI summarize only after messy notes exist.** If there is no rough trail, the summary is not yours — it is a borrowed deliverable.

# 53. Engagement quality - How well did you collaborate with AI (while learning)?

Use conversational AI to learn, plan, critique, and reflect. You set boundaries, cite AI contributions, and demonstrate growth over time.

We **require** conversational AI use and evidence of process — not avoidance. For LO6, **"AI" means conversational AI** (ChatGPT, Claude, Copilot), not building models (that is a different learning outcome).

## 53.1 Two students revisited

Same deliverable, different growth. Student A **accepted** output; Student B **integrated** AI with their own thinking and left a trail. Assessment must look beyond polish:

> If we only assess the document, we assess ChatGPT — not the student.

The only way to know whether someone understands their work is **ongoing conversation** across tasks and time. AI makes skipping that conversation impossible.

## 53.2 Four levels of engagement

These describe **processes**, not fixed identities. Someone may Integrate on Monday and Accept on Tuesday — that is fine if they **notice** and recover when the task demands more.

| Level | Name | Investment | What it looks like |
|---|---|---|---|
| 1 | **Accepting** | Minimal | Taking output at face value; copy-paste with surface edits |
| 2 | **Refining** | Moderate | Adjusting and polishing; core ideas unchanged |
| 3 | **Challenging** | Significant | Pushing back, verifying, fact-checking, documenting disagreements |
| 4 | **Integrating** | Deep | Combining AI input with own thinking; original insights emerge |

**Calibrating:** A student who says they were *Challenging* because they "Googled whether numbers seemed reasonable" may be *Refining with good intentions*. Help them see what actual Challenging would require — that is learning, not punishment.

Know when Accepting is enough (quick fact lookup) and when Integrating is required (complex analysis). Be honest about the difference.

# 54. Coaching AI collaboration

| Pattern | What you observe | What may be happening | Try asking |
|---|---|---|---|
| Hiding use | Polish, vague "I used AI a bit", can't explain reasoning | Shame, fear, or Accepting without awareness | Walk me through your process — where did *you* decide? |
| Receipts only | Long prompt logs, no reflection | Documentation mistaken for engagement | Where did you disagree or learn something new? |
| Avoiding AI | Struggle on tasks AI could scaffold | AI seen as cheating or threatening | What if AI challenged your ideas instead of writing them? |
| Growing collaboration | Dialogue trail; can separate contributions | This is what we want | What made you push back on X? |
| Regressing under pressure | Early Integrating, late Accepting when rushed | Deadlines; human, but needs noticing | What happened? How might you handle that differently next time? |
| Outsourcing the thinking | Polished portfolio or report; cannot recall or explain own text after time away | Accepting at scale — deliverable done, learning skipped | What do you still own? What would recovery look like (messy notes first, AI as memory not author)? |

You do not need to be brilliant at prompting. You need to be **in the driver's seat**.

# 55. AI augmented learning: some takeaways

1. **Quality over quantity** — how well you collaborated, not how much AI they used
2. **Process over polish** — dialogue documentation beats glossy output
3. **Growth is messy**

# 56. Coen's personal toolchain: Lepiter, paper, and LLM

Mainly for Coen's own reference. (Copied together… not human-readable … yet … well, maybe)

## 56.1 Lepiter: a living wiki

Inside Glamorous Toolkit (GT) — a moldable development environment — using **Lepiter**, its built-in knowledge-management system. Each page is assembled from **snippets** (Markdown text, live code, queries, visualizations), and pages link into a **knowledge graph** you can browse over time.

Lepiter is not a polished publishing tool. It is a **persistent, linkable workshop**.

## 56.2 Paper bullets when load is high

When meetings stack up or attention is thin, I reach for a **paper notebook** and **bullet points** — fast, low ceremony, no layout trap.

## 56.3 The combination principle

No single medium wins every cognitive job. A practical rule:

| Cognitive job | Favored medium | AI role |
|---|---|---|
| **Structure while learning** | Lepiter pages + links | Critique gaps later — not first author |
| **Capture under load** (back-to-back meetings, travel) | Paper bullets | Transcribe or summarize only after you have rough bullets |
| **Polish or archive** (portfolio prose, colleague-facing write-up) | Your home base (Lepiter, repo notes, or docs) | Tighten wording, spot missing stakeholders — not replace your trail |

## 56.4 Process visibility, not “fancy IDE”

**process visibility** — making thinking and exploration inspectable — not “another IDE to learn.” Lepiter shows *how* someone structures a problem over days: linked notes, live snippets, revisitable reasoning. That parallels what we want from AI-augmented learning: a trail you can explain, not only a deliverable that looks finished.

## 56.5 GT and The GT Book

**learning in the same environment you think in**.

- GToolkit Book: embedded in GToolkit itself.

## 56.6 Tips for colleagues

These transfer without installing Glamorous Toolkit:

1. **Separate capture tools from polish tools.** A frictionless inbox (paper, voice memo, scratch file) and a slower “home” (wiki, repo docs, Obsidian, team handbook) beat one chat window that does both — chats optimize for fluent answers, not your long-term memory.
2. **Pick one home for notes that must outlive a session.** Where will *you* find this in three months? If the answer is “scroll the chat,” the encoding step probably did not happen.
3. **When speed beats perfection, use paper.** Bullets in a notebook during a dense day are a feature, not a failure of discipline — migrate to digital structure when load drops, then involve AI if you need critique or prose.

# 57. References AI & Education


- students-should-only-use-ai-for-things...
- The Objective of Education Is Learning, Not Teaching
- 202606: bron.fontys: Mogen studenten zomaar AI gebruiken?
- misschien-begrijpen-ze-niet-elke-regel-code-en-dat-is-oke
- podcast: Your undivided Attention: AI Is Breaking Education. Rebecca Winthrop Has the Blueprint to Fix It.
- 202603: JP Ligthart: De verschuiving in softwareontwikkeling door AI: van implementatie naar oordeel
- 202512: Cursor: onderwijs aan TU/e verandert fundamenteel
- bron: word-geen-ai-zombie-zo-blijf-je-kritisch-in-een-wereld-vol-ai
- bron: ICT maakt eigen 'zelf in te kleuren' AI-opleiding mogelijk
- Saçan: Schuurpapier voor het onderwijs...
- bron.fontys.nl/nieuw-fraudebeleid-met-focus-op-preventie
- How AI is changing education
- column-mark-de-graaf-ga-ict-studeren
- bron.fontys: een-eigen-ai-tool-voor-fontys
- Three things chess can teach us...

# 58. Tools, best practices & lesson material

- tintara.nl
- aivoordocenten.nl
- EduGenai (Npuls)
- How to cite ChatGPT? Use AI Archive
- aiarchives.org
- You did it together with AI? Make a statement!
- hbo-i-outcomes-example-generator chatbot
- https://roadmap.sh/ai

# 59. AI voor docenten

aivoordocenten.nl

# Part VII

# AI & democracy / journalism / dangers

# 60. AI, Journalism, Democracy

- 2026-06 NOS: sociale-media-vaak-gebruikt-maar-niet-vertrouwd-als-bron-van-nieuws
- NOS: zorgen-over-trollenlegers-en-andere-inmenging-rond-verkiezingen
- NOS: Special: hoe herken je valse beelden over de oorlog?
- NOS: Lokale partijen worden genegeerd door AI-chatbots, 'niet geschikt als stemhulp'
- Jonathan Boymal: AI's deepest educational risk is cognitive offloading…
- De Speld: ai-beelden-van-beverwijk-misleiden-toeristen
- Shuwey Fang: information-ecosystem-being-redrawn-ai-might-be-good-news
- stichtingrpo.nl: introductie-ai-kompas
- Hey Aftonbladet (chatbot): What do YOU want to know?
- Fake disease Bixonimania

# 61. Why Silicon Valley Wants to Leave Humanity Behind

Your Undivided Attention: Why Silicon Valley Wants to Leave Humanity Behind

# 62. Post mortem for Huggingface hack

redwood research

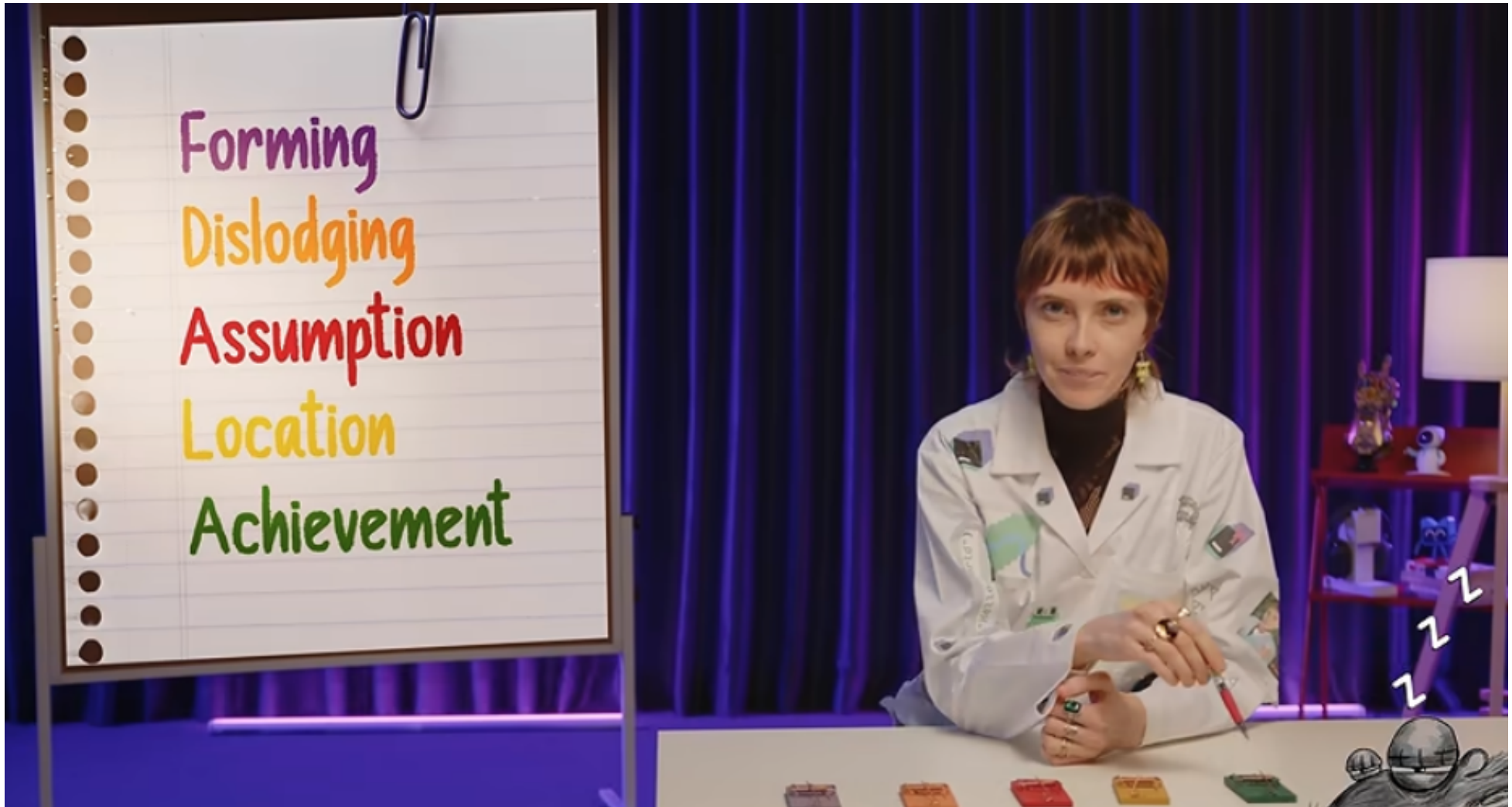
# Part VIII

# AI & (learning) programming & Software Engineering

# 63. What about IT jobs in the future?

eenvandaag.avrotros.nl : jongeren-kiezen-vanwege-ai-minder-vaak-voor-opleiding-tot-icter-terwijl-we-die-juist-hard-nodig-hebben

## 64. Therapy for the Vibe-Coded Brain



video Therapy for the Vibe-Coded Brain

# 65. Coding with AI

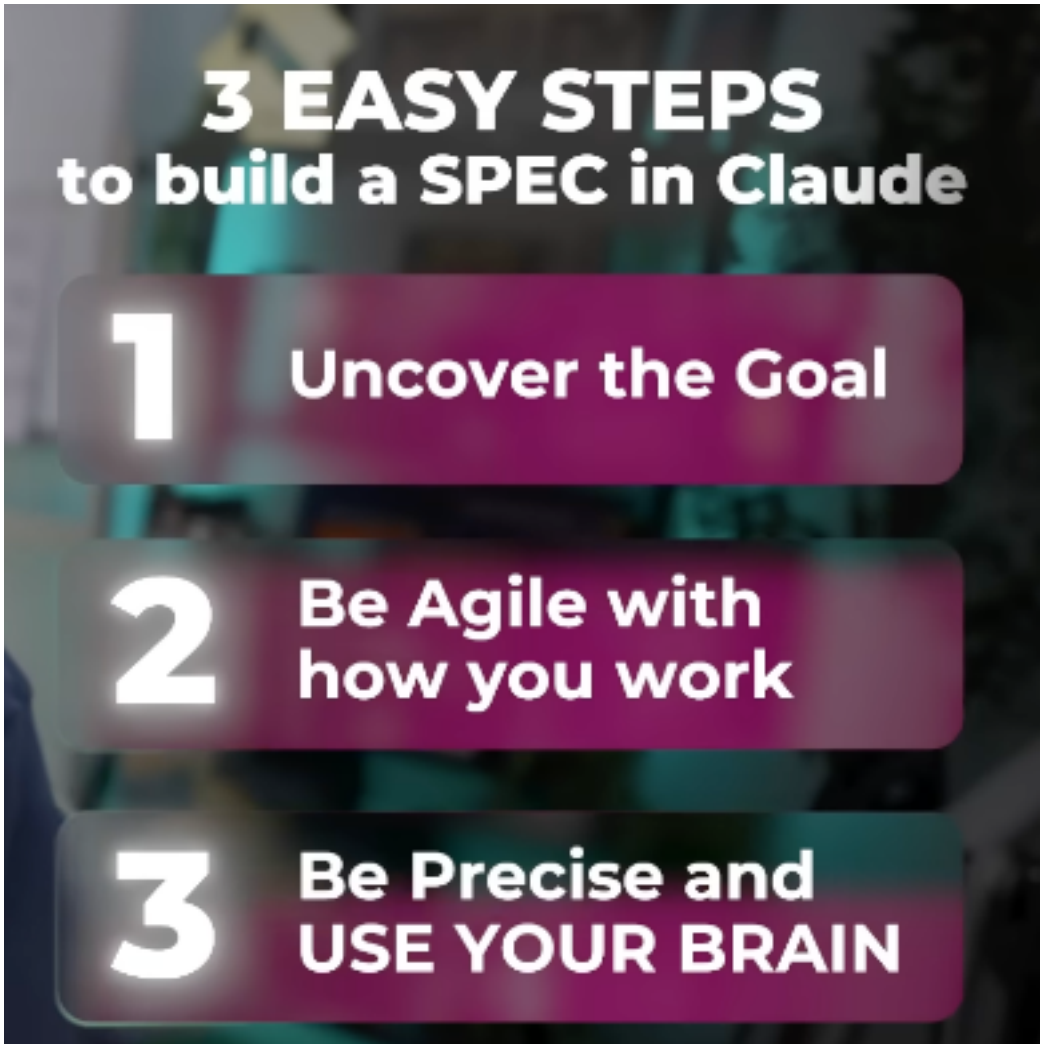
Master the art of providing **context**.

- Cursor
- 202510 LLM Coding Personalities
- Anthropic about Context
- Most devs don't understand how context windows work

# 66. Coding with GenAI

- cursor course: AI Fundamentals
- vs code with co-pilot (free plan)
- Cursor.com (20 euro p/m)
- AIDER.chat (free)
- Open Devin: Create any Application with Open Source AI Engineer
- Avante (AI in neovim, free)

## 67. Karpathy's method



youtube: Stop prompting Claude. Use Karpathys's Method instead

# 68. DSPy: Let the LLM Write the Prompts

- Let the LLM Write the Prompts: An Intro to DSPy in Compound AI Pipelines

# 69. Spec-Drive Development with Coding Agents

Short course at deeplearning.ai:

Spec-Drive Development with Coding Agents

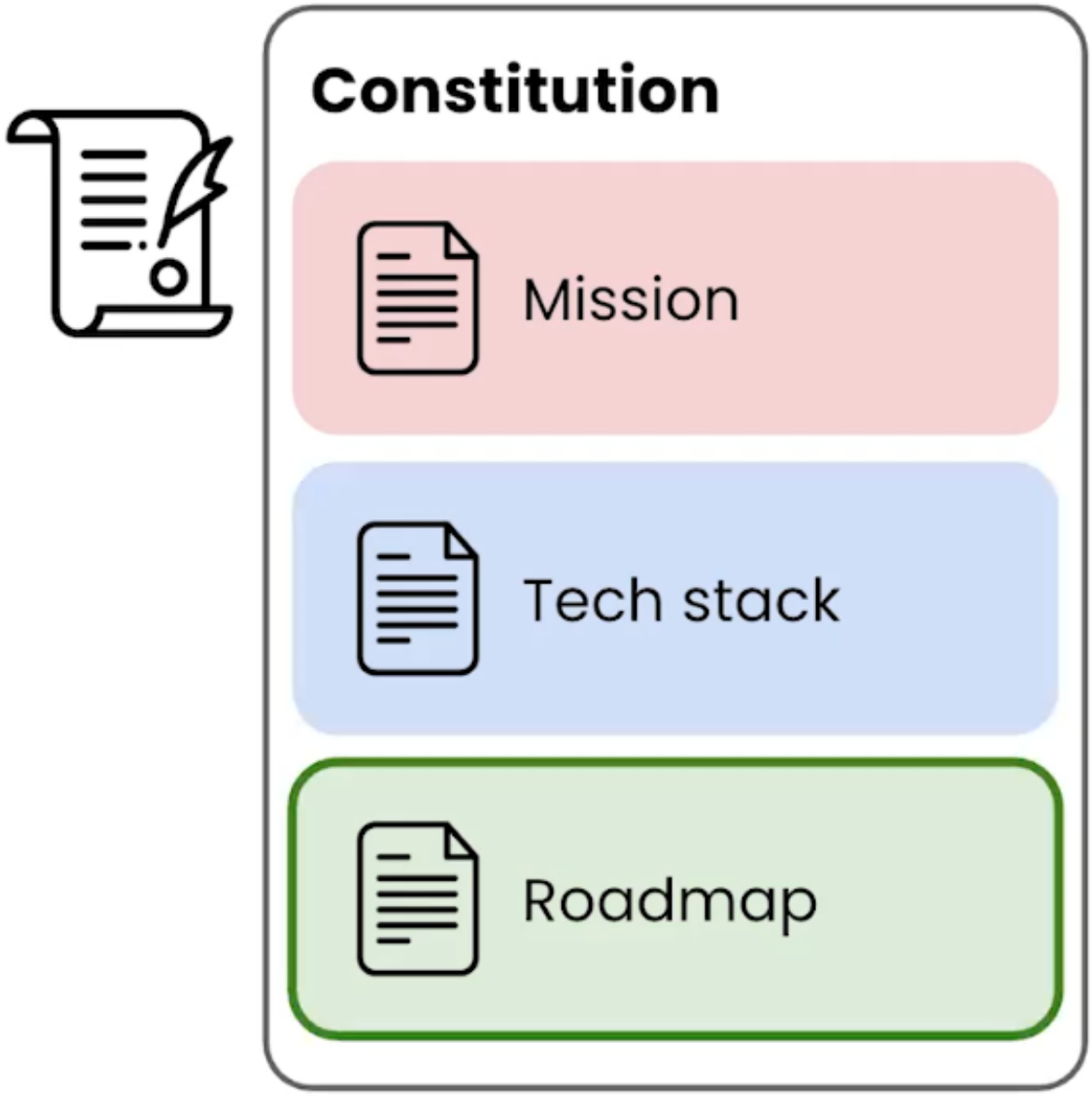



Figure 69.1: Constitution

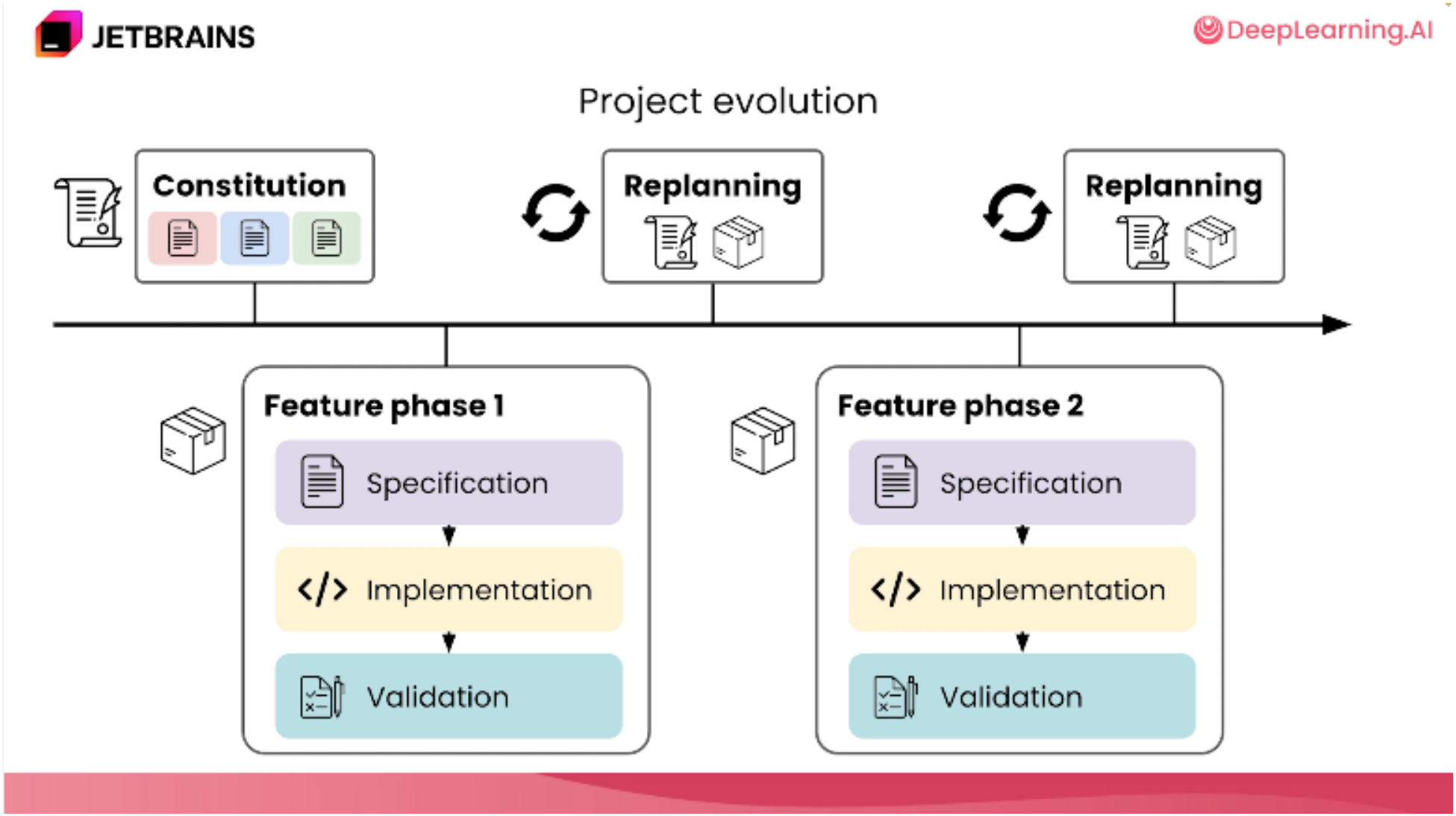



Figure 69.2: Project evolution

But if you really want to dive in ask your AI-coding-tool to do ***Spec Driven Development***.

# 70. Software Engineering with AI

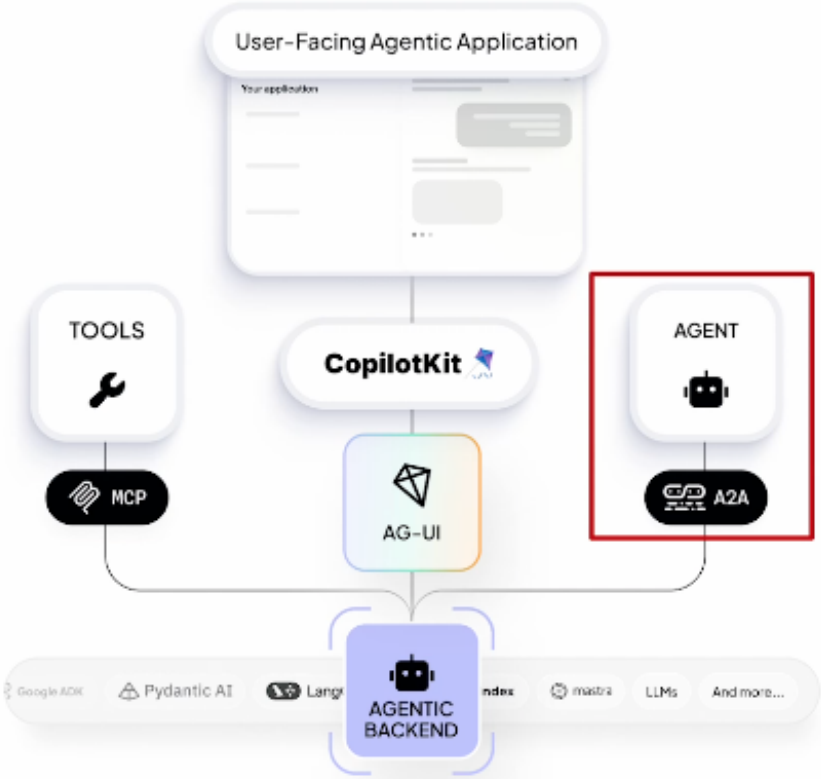
- 202606, Heck: competences-for-the-ai-augmented-software-engineer
- NL: kwaliteit-software-onder-druk-door-ai-gegenereerde-code
- 202511 - Martin Fowler: How AI will change software engineering
- Radical Object Orientation in the Age of AI Code Generation by Ralf Westphal #AgileIndia 2025

# 71. Generative UI

Course at deeplearning.ai:

- build-interactive-agents-with-generative-ui

# CopilotKit & AG-UI: The Agentic Frontend Stack




- CopilotKit:
  - Open-Source Developer SDKs
  - Cloud for building agentic applications
  - Purpose built for the agentic paradigm
- AG-UI (The Agent-User Interaction Protocol):
  - Horizontal standard connecting agentic backends & agentic frontends

CopilotKit

DeepLearning.AI

# 72. Jobs and AI

July 2025

- ai-isn-t-coming-for-your-job-it-s-coming-for-your-mind
- FD: ai vervangt de programmeur nog niet
- pabo-wint-aan-populariteit-ict-en-fysio-juist-niet
- UWV: Kansrijke beroepen 2025-2026

## 72.1 it-s-coming-for-your-mind

'…AI is not culturally neutral in what it transmits. Research published in Nature Human Behaviour found that when AI is prompted in Chinese versus English, it exhibits systematically different cultural orientations, more interdependent and holistic in Chinese, more independent and analytic in English.'

'Professional bodies, universities and employers need to preserve the training pathways that build genuine expertise, even when AI makes them look slow and inefficient, and then teach AI orchestration as an advanced capability that sits on top of that foundation. Some are already doing this.'

# Part IX

# AI Cookbook

# 73. Transcribing / Speech2Text

Talking to your machine!

- github.com/cjpais/Handy: open source Speech2Text
- Transcribbly
- elevenlabs.io/audio-to-text
- Otter.ai

**Otter.ai : Your Mission (in 3 steps):**

1. **Sign Up:** Go to `Otter.ai` and create a free account. ✍️
2. **Upload:** Find the "Import" button and upload your meeting audio file (`.mp3`, `.wav`, etc.). 📤
3. **Transcribe:** Otter will work its magic! ✨ Then you can read, edit, and export your new transcript. 📄

**That's it!** You've just transcribed a meeting with AI. 🙌🚀

Be aware of your privacy when you use an online, free tool!

# 74. Scenario: Summarize a Text

Let's make a long story short with AI!

We'll use a simple online tool called **Summarizer**.

**Your Mission (in 3 steps):**

1. **Find Text:** Grab a long article or text you want to shorten.
2. **Paste & Go:** Copy the text, paste it into the Summarizer website, and click the "Summarize" button.
3. **Read:** Enjoy your new, shorter text! You can even choose how long you want the summary to be.

**That's it!** You've just summarized a text with AI. So cool!

# 75. Scenario: Replying to an Email

Let's see context in action.

**Bad Prompt (No Context)**

> "Draft a polite and professional email saying I can't make it."

The AI will produce a generic, fill-in-the-blanks template. It's not very helpful because it lacks any specific details.

**Good Prompt (With Context)** > "I need to reply to this email. [**Paste the full text of the original email here**]. Please draft a polite and professional reply explaining that I can't make the 'Project Phoenix' meeting on Wednesday because of a conflict, but I am very interested. Ask if they can send me the minutes and suggest I'm available to connect next week."

See the difference? By providing the original email and clear instructions, you've given the AI all the context it needs to draft a perfect, ready-to-send reply.

**The "Context Window": An AI's Short-Term Memory**

It's important to know that every AI has a limited memory, called a **"context window."** This is the maximum amount of information (your prompt, the text you've pasted, the conversation history) that the model can "see" at one time.

If your conversation gets very long, the AI might start to "forget" things you discussed at the beginning. If you notice the AI losing track, it might be time to start a new conversation to give it a fresh, clean context to work with.

# 76. Create an app with AI

- creating-an-app-with-ai

# Part X

# Train, Fine Tune, RAG, Validate

# 77. Train, Fine Tune, RAG

Several ways to ‘teach’ the AI about the knowledge it needs to perform the task you need it for. The most easy of these is building a RAG system: Retrieval Augmented Generation.

# 78. RAG: Retrieval Augmented Generation

- deeplearning.ai course: Chat with your data
- building-evaluating-advanced-rag
- IBM, March 2026: Is RAG still needed?
- IBM, March 2026: What is OpenRAG?
- IBM, Marina Danilevsky (7 min)
- VideoRAG: Chat with Your Videos

# 79. Finetune

- AI Engineering at Jane Street - John Crepezzi
- xcancel introducing qqWen

# 80. Training

Training a model from scratch is a complex and resource-intensive process. It involves collecting a large dataset, preprocessing the data, and training the model using powerful hardware. This is typically done by large organizations with significant resources.

short course: fine tuning

# 81. Visual Recognition

CLIP-models, dyno, yolo, resnet, alexnet.

# 82. Evaluation / Validation

***Evaluate*** means assess *how good*, *effective*, or *suitable* something is.

***Validate*** means check that something is correct or meets defined requirements.

For example, with an AI model:

- Evaluate: How accurate or helpful is it across a set of tasks?
- Validate: Does it meet the agreed launch criteria, such as a minimum accuracy or safety threshold?

They overlap: validation often uses evaluation results. The distinction is usually the purpose—evaluation helps you understand performance; validation checks whether it satisfies a specific requirement.

- short course deeplearning.ai automated-testing-llmops
- James Bach - Seriously Testing LLMs

# Part XI

# Neural Network, How does it all work?

# 83. If you prefer a story...

The story that follows right here explains the ideas behind a Neural Network from a technical perspective. If you would rather read an Instructive Story, a Saga, read it online at: Lonn's neural-net-saga. Scroll down a bit and start reading 'The Percy Chronicles: A Neural Network Saga'. At the end of that story you will find some python to get hands-on with.

# 84. Understanding the Perceptron

## 84.1 The Biological Inspiration: From Brain Neurons to Artificial Intelligence

The Perceptron represents one of the most fundamental concepts in artificial intelligence, drawing its inspiration directly from the human brain's neural structure. This groundbreaking idea was first introduced in 1943 by Warren S. McCulloch and Walter Pitts in their seminal paper 'A Logical Calculus of the Ideas Immanent in Nervous Activity', where they proposed a mathematical model of biological neurons.

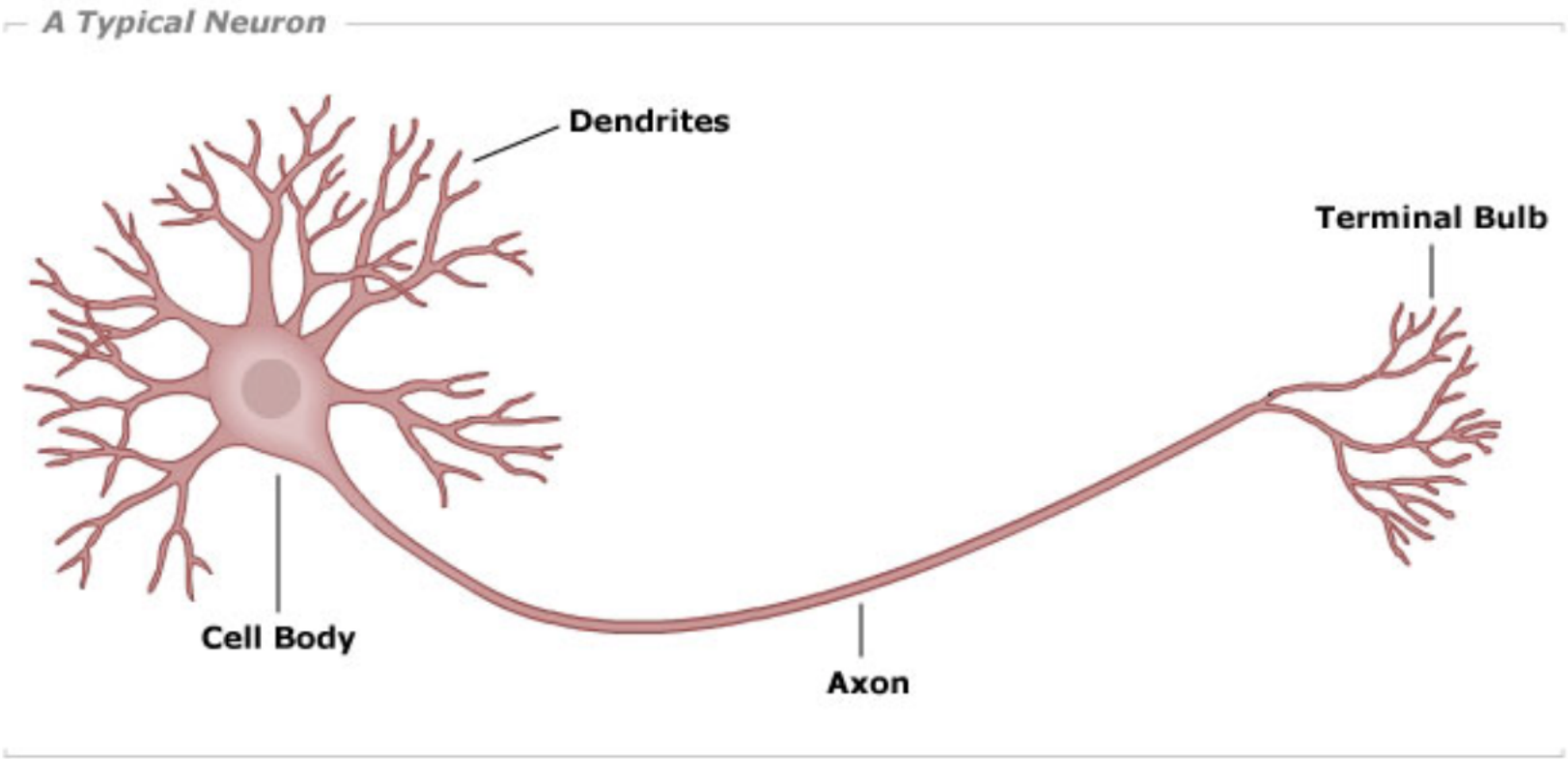



Figure 84.1: A typical biological neuron structure

## 84.2 From Biology to Machine: Implementing a Perceptron

A Perceptron's architecture mirrors its biological counterpart through three key components: `inputs`, `weights`, and a `bias`. Each input connection has an associated weight that determines its relative importance, while the bias helps adjust the Perceptron's overall sensitivity to activation.

Let's look at a simple yet useful perceptron with 2 inputs.

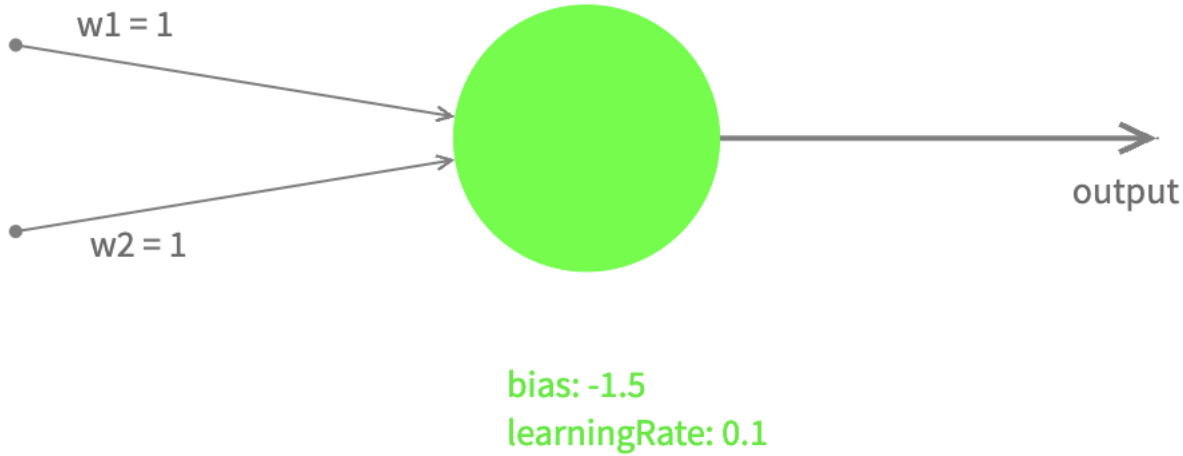



Figure 84.2: Perceptron's architectural diagram

We'll call our input values `x1`, `x2` with their corresponding weights `w1`, `w2`. The Perceptron processes these inputs in two steps:

1. First, it calculates a `weighted sum` and adds the bias: `z := w1*x1 + w2*x2 + bias`
2. Then, it applies what we call an **activation function** to produce the final output: let's use a very simpel activation function, called a Step function:

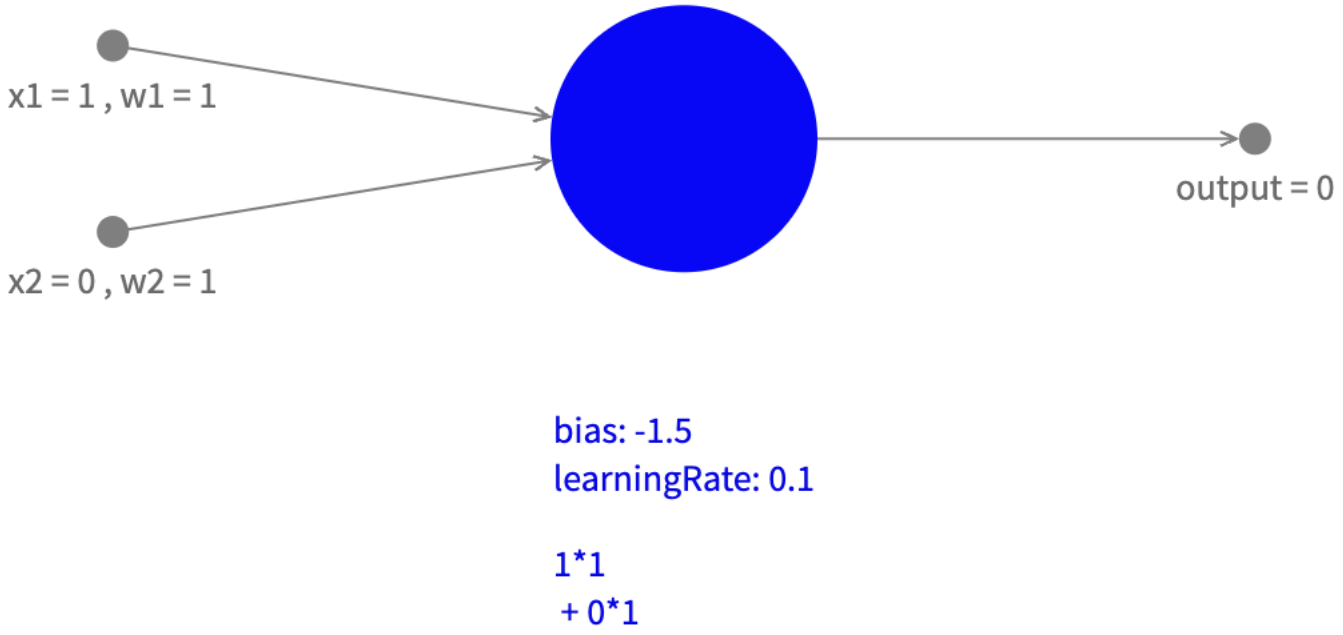



Figure 84.3: Perceptron's architectural diagram

$$\begin{cases} \text{Output is 1 if } z > 0 \\ \text{Output is 0 if } z \leq 0 \end{cases}$$

which determines the final output.

Let's restrict ourselves for now to possible input values 0 and 1: If we look at all possibilities combinations of input and the corresponding output we can create a table:

| Input 1 | Input 2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

A close look will tell us that the output is only 1 when inputs are 1, and 0 in all other cases, which you could recognize as a logical AND. So with these weights and bias this Perceptron can be used to act as a logical AND.

For different values it will behave like a logical OR (and more). Can you come up with those values?

## 84.3 Network

By combining several Perceptrons (sending the output of a perceptron to the input of another one) you can probably imagine that it is possible to create Networks of Perceptrons. By changing the values of weights and biases of the connected Perceptrons it is possible to build complex electronic circuits.

When we generalize this concept to other values, not only 0 and 1, and different activation functions, the Perceptron becomes an incredibly versatile tool. This generalization opens up possibilities for pattern recognition, classification tasks, regression problems, and complex decision-making systems. This is where the true power of neural networks begins to emerge, as they can learn to handle continuous data and make sophisticated decisions based on multiple inputs.

Up until now we didn't look at how a perceptron can learn and become smarter. That will be subject of next chapter chapters. The concept of a Perceptron was generalized to what we now call an (artificial) Neuron.

Search terms: Perceptron, Artificial Neuron, `Multi Layered Perceptron` (MLP), `(Artificial) Neural Network` (ANN).

## 84.4 First Implementation of Perceptron algorithm

According to Wikipedia:

> The artificial neuron network was invented in 1943 by Warren McCulloch and Walter Pitts in 'A logical calculus of the ideas immanent in nervous activity'. the Perceptron Machine was first implemented in hardware in the Mark I, which was demonstrated in 1960.
>
> It was connected to a camera with 20×20 cadmium sulfide photocells to make a 400-pixel image. The main visible feature is the sensory-to-association plugboard, which sets different combinations of input features. To the right are arrays of potentiometers that implemented the adaptive weights.

## 84.5 Reference

- wikipedia: perceptron

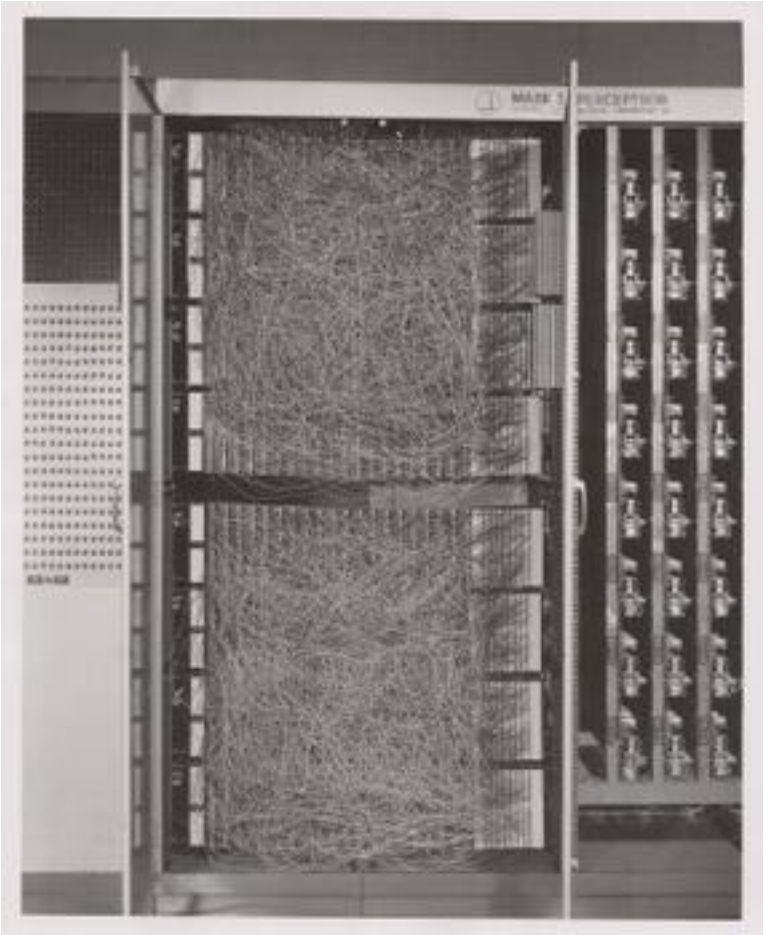


Figure 84.4: The Mark I Perceptron machine, the first implementation of the perceptron algorithm (source: wikipedia)

# 85. The Learning Perceptron

One of the most fascinating aspects of Perceptrons is their ability to learn from examples. Instead of manually setting weights and bias, we can train a Perceptron to discover the optimal parameters through a process called supervised learning.

## 85.1 The Learning Algorithm

The learning process follows these key steps:

1. Start with random weights and bias
2. Present a training example
3. Compare the Perceptron's output with the desired output
4. Adjust the weights and bias based on the error
5. Repeat with more examples until performance is satisfactory

### 85.1.1 Mathematical Foundation

The weight update rule is elegantly simple:

```
new_weight := current_weight + learning_rate * error * input
```

Where:

- `learning_rate` is a small number (like 0.1) that controls how big each adjustment is
- `error` is the difference between desired and actual output (1 or -1)
- `input` is the input value for that weight

### 85.1.2 Training Process

To train the Perceptron, we have to have `labeled data` (ie. input data combined with the desired output for those values)

So for training AND gate behavior we have to list all combinations of 2 bits that are possible as input, and also the desired output value:

```
| Input  | Desired Output |
|--------|----------------|
| (0, 0) | 0              |
| (0, 1) | 0              |
| (1, 0) | 0              |
| (1, 1) | 1              |
```

and training (1 epoc) means calling the train function with each of these examples:

```
foreach dataItem in trainingData do:
    inputs := dataItem[0]
    desiredOutput := dataItem[1]
    learningPerceptron train(inputs, desiredOutput)
```

## 85.2 Visualizing the Learning Process

As the Perceptron learns, its decision boundary gradually moves to the correct position. You can monitor this progress by:

1. Tracking the error rate over time
2. Visualizing the decision boundary's movement
3. Testing the Perceptron with new examples

## 85.3 Practical Considerations

For successful learning: - Ensure your training data is representative - Consider using multiple training epochs (complete passes through the data) - Monitor for convergence (when the weights stabilize) - Be aware that not all problems are linearly separable

In the next chapter, we'll explore the limitations of what a single Perceptron can learn, which will lead us naturally to the need for more complex neural networks.

# 86. Understanding Perceptron Limitations

## 86.1 The XOR Problem: A Classic Challenge

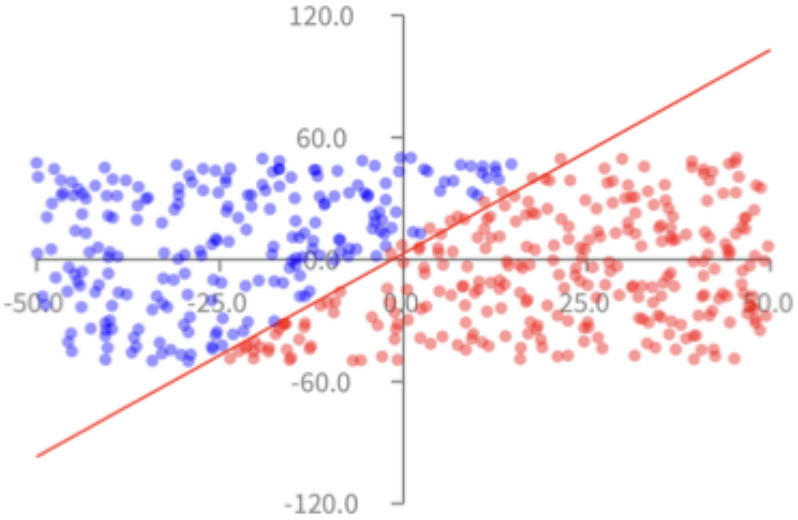
While Perceptrons are powerful tools for many classification tasks, they face a fundamental limitation: they can only solve linearly separable problems. The classic example of this limitation is the XOR (exclusive OR) function.

### 86.1.1 What is XOR?

The XOR function outputs 1 only when exactly one of its inputs is 1: - Input (0,0) → Output: 0 - Input (0,1) → Output: 1 - Input (1,0) → Output: 1 - Input (1,1) → Output: 0

# What we have seen so far

We have seen that the perceptron can (more or less accurately) guess the side on which a point is located




34

Figure 86.1: Visual representation of XOR problem

### 86.1.2 Why Can't a Single Perceptron Solve XOR?

A Perceptron creates a single straight line (or hyperplane in higher dimensions) to separate its outputs. However, the XOR problem requires two separate lines to correctly classify all points.

# What we have seen so far

We can easily make our perceptron to represent the AND, OR logical operations

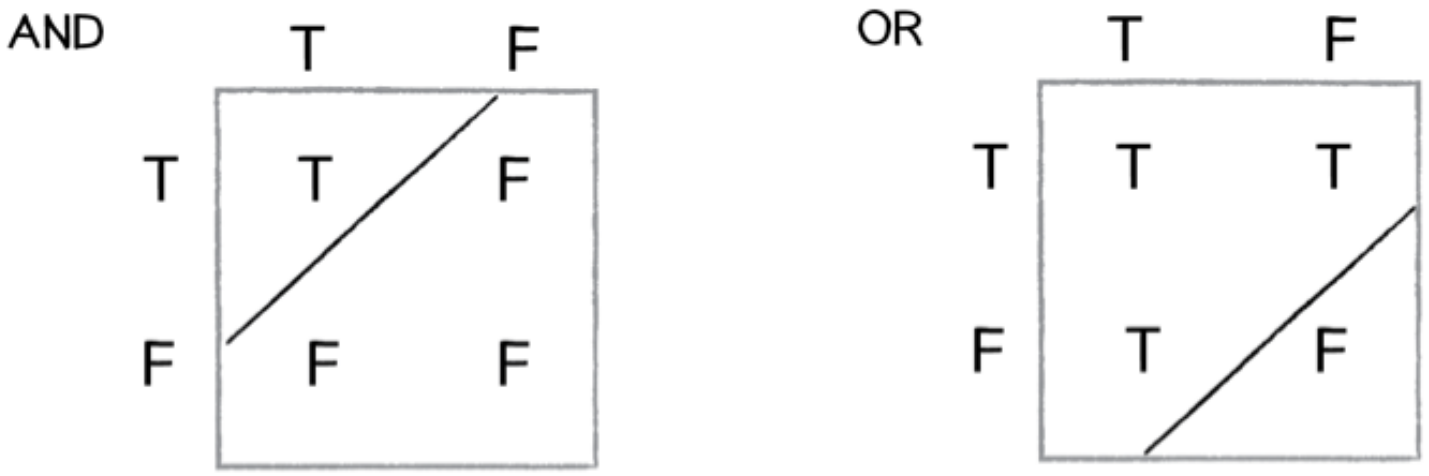





Figure 86.2: Attempted linear separation of XOR

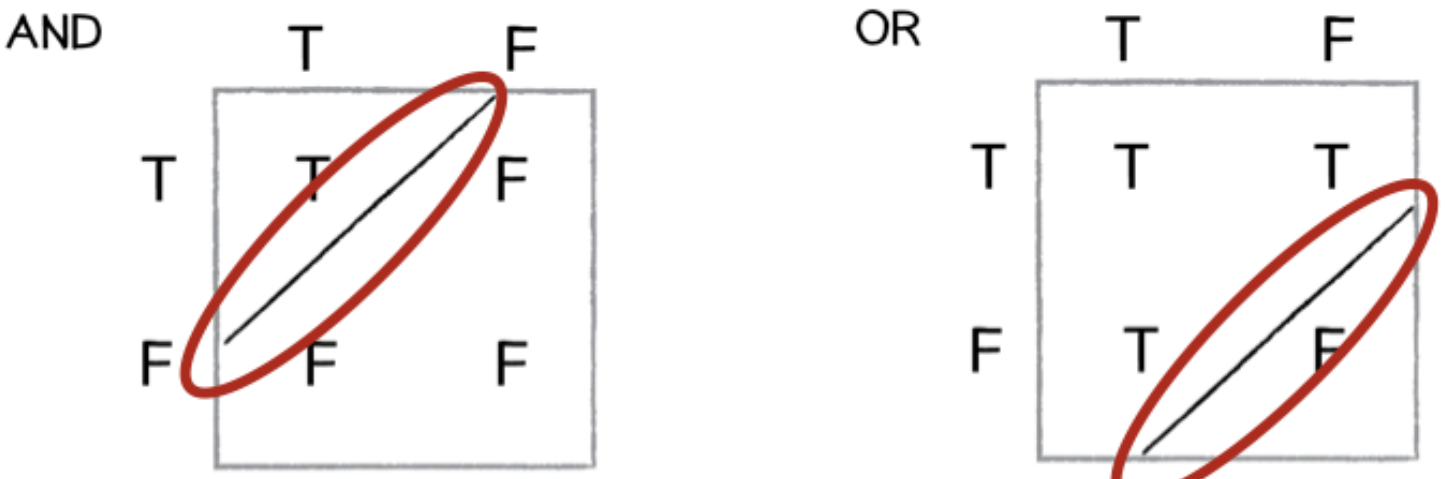
As you can see, no single straight line can separate the points where output should be 1 (blue) from points where output should be 0 (red).

## 86.2 The Solution: Multiple Layers

To solve the XOR problem, we need to combine multiple Perceptrons in layers. This is our first glimpse at why we need neural networks!

# What we have seen so far

We can easily make our perceptron to represent the AND, OR logical operations



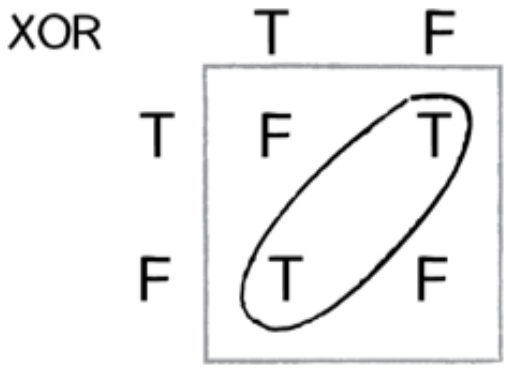

This is very similar to the space & point problem.
It is all about having a line as a limit

39

Figure 86.3: Multi-layer solution

By using multiple Perceptrons, we can: 1. First create separate regions with individual Perceptrons 2. Then combine these regions to form more complex decision boundaries

# Limitation of a perceptron




With the XOR operation, you cannot have one unique line that limit the range of true and false



Figure 86.4: Complete neural network solution

## 86.3 Key Takeaways

1. Single Perceptrons can only solve linearly separable problems
2. Many real-world problems (like XOR) are not linearly separable
3. Combining Perceptrons into networks overcomes this limitation
4. This limitation led to the development of multi-layer neural networks

In the next section, we'll explore how to build and train these more powerful multi-layer networks.

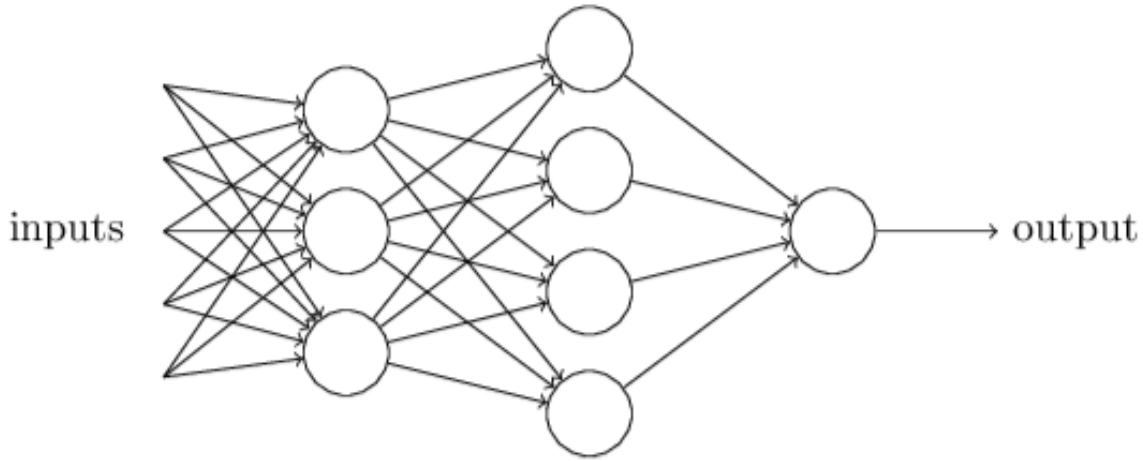
# 87. Introduction to Neural Networks

## 87.1 Beyond Single Perceptrons: Building Neural Networks

Having seen the limitations of single Perceptrons, we now venture into the fascinating world of neural networks. These powerful structures combine multiple Perceptrons in layers to solve complex problems that single Perceptrons cannot handle.

# Network of neurons

A network has the following structure




41

Figure 87.1: Basic neural network architecture

## 87.2 Understanding Network Architecture

A typical neural network consists of three main components:

1. **Input Layer**: Receives the raw data
2. **Hidden Layer(s)**: Processes the information through multiple Perceptrons
3. **Output Layer**: Produces the final result

## 87.3 Key Components

Each connection in the network has: - A weight that determines its strength - A direction of information flow (forward only) - An associated neuron that processes the incoming signals

## 87.4 How Information Flows

The network processes information in these steps:

1. Input values are presented to the input layer
2. Each neuron in subsequent layers:
    - Receives weighted inputs from the previous layer
    - Applies its activation function
    - Passes the result to the next layer
3. The output layer produces the final result

## 87.5 Creating a Simple Network

You probably have seen a picture of a neural network before.

Neural Networks can

1. solve problems that are more difficult.
2. Handle complex pattern recognition
3. Learn hierarchical features automatically
4. Scale well to large problems

In the next sections, we'll explore practical applications and see how to train networks on real-world data.

# 88. Practical Example: Classifying Iris Flowers

## 88.1 A Real-World Machine Learning Challenge

The Iris flower classification problem is a classic example in machine learning. It involves predicting the species of an Iris flower based on measurements of its physical characteristics. This problem perfectly illustrates how neural networks can solve real-world classification tasks.






Figure 88.1: Different types of Iris flowers

## 88.2 The Dataset

The Iris dataset includes measurements of three different Iris species: - Iris Setosa - Iris Versicolor - Iris Virginica

For each flower, we have four measurements: 1. Sepal length 2. Sepal width 3. Petal length 4. Petal width

Building a network that can do this is really outside of the scope of these notes, but a lot of info can be found on the internet on `Iris Classification`.

## 88.3 Key Learning Points

1. Neural networks can handle multi-class classification
2. Real-world data often needs preprocessing
3. We can measure success with accuracy metrics
4. The same principles apply to many similar problems

This practical example demonstrates how neural networks can solve real classification problems. In the next section, we'll explore the mathematics behind how these networks learn.

# 89. The Mathematics Behind Neural Networks

## 89.1 Understanding the Magic

While neural networks might seem magical, they're built on solid mathematical foundations. Let's demystify (a bit of) how they actually work under the hood.

## 89.2 The Building Blocks

### 89.2.1 1. Neurons and Weights

To be formally correct we should say `artificial neuron` to distinguish them from `biological neurons` like we have in our brain. A neuron normally has inputs: 1, or 2, or $\cdots$

Each neuron performs two key operations: 1. Weighted sum of inputs. 2. Activation function: $a = f(z)$

### 89.2.2 2. Activation Functions

Common activation functions include:

1. Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
   - Outputs between 0 and 1
   - Useful for probability predictions
2. ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$
   - Simple and efficient
   - Helps prevent vanishing gradients
3. Tanh: $f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
   - Outputs between -1 and 1
   - Often better than sigmoid for hidden layers

## 89.3 The Learning Process

### 89.3.1 1. Forward Propagation

Information flows through the network.

### 89.3.2 2. Loss Calculation

Measure the network's Error and Backpropagation

- What is the output?
- What would be my desired output?

The smaller the difference between the output I got and the output I desired, the better the output of my model is. This difference is calculated with a so-called Loss function. Backpropagation is an algorithm that helps make that difference small. When backpropagation is performed we call that Training the AI model.

# 90. Exploring Neural Network Architectures

## 90.1 The Rich Landscape of Neural Networks

Neural networks come in many shapes and sizes, each designed to excel at specific types of tasks. Let's explore some of the most important architectures and their applications.

## 90.2 Feedforward Neural Networks (FNN)

The classic architecture: Information flows in one direction:

- Input layer → Hidden layer(s) → Output layer
- Perfect for classification and regression tasks
- Examples: Our Iris classifier, handwriting recognition

## 90.3 Convolutional Neural Networks (CNN)

Inspired by the human visual cortex:

- Specialized for processing grid-like data (images, video)
- Uses convolution operations to detect patterns
- Excellent at feature extraction
- Applications: Image recognition, computer vision, medical imaging

## 90.4 Recurrent Neural Networks (RNN)

Networks with memory:

- Can process sequences of data

- Information cycles through the network
- Great for time-series data and natural language
- Applications: Language translation, speech recognition, stock prediction

## 90.5 Long Short-Term Memory (LSTM)

A sophisticated type of RNN:

- Better at remembering long-term dependencies
- Controls information flow with gates
- Solves the vanishing gradient problem
- Applications: Text generation, music composition

## 90.6 Autoencoders

Self-learning networks:

- Learn to compress and reconstruct data
- Useful for dimensionality reduction
- Can detect anomalies
- Applications: Data compression, noise reduction, feature learning

## 90.7 Generative Adversarial Networks (GAN)

Two networks competing with each other:

- Generator creates fake data
- Discriminator tries to spot fakes
- Through competition, both improve
- Applications: Creating realistic images, style transfer, data augmentation

## 90.8 Choosing the Right Architecture

The choice of architecture depends on:

1. Type of data (images, text, time-series)
2. Task requirements (classification, generation, prediction)
3. Available computational resources
4. Need for real-time processing

## 90.9 Future Directions

Neural network architectures continue to evolve:

- Hybrid architectures combining multiple types
- More efficient training methods
- Better handling of uncertainty
- Integration with other AI techniques

In the next section, we'll dive deeper into training these networks effectively.

# 91. Transformers

deeplearning.ai - transformers-in-practice oct2025 - Everything about Transformers

FSM: Finite State Machine

# 92. Resources and References AI

## 92.1 Books

1. **Neural Networks and Deep Learning**
    - Author: Michael Nielsen
    - Free Online Book
    - Perfect for beginners and intermediate learners
    - Clear explanations with interactive examples
2. **Deep Learning**
    - Authors: Ian Goodfellow, Yoshua Bengio, Aaron Courville
    - Available Online
    - Comprehensive coverage of deep learning
    - Industry standard reference
3. **Agile AI in Pharo**
    - Author: Alexandre Bergel
    - Practical implementation in Pharo
    - Hands-on examples and exercises
    - Book Link

## 92.2 Video Courses and Tutorials

### 92.2.1 1. Foundational Series

- 3Blue1Brown Neural Networks
    - Visual explanations

  - Mathematical intuition
  - Clear animations

https://www.youtube.com/watch?v=O5xeyoRL95U

#### 92.2.2 2. Programming Tutorials

- Fast.ai Deep Learning Course
  - Practical approach
  - Top-down learning
  - Real-world applications

#### 92.2.3 3. Advanced Topics

- Stanford CS231n
  - Computer Vision
  - Deep Learning
  - State-of-the-art techniques

### 92.3 Online Platforms

#### 92.3.1 1. Interactive Learning

- Kaggle Learn
  - Hands-on exercises
  - Real datasets
  - Community support

#### 92.3.2 2. Research Papers

- arXiv Machine Learning
  - Latest research
  - Open access
  - Preprint server

### 92.4 Community Resources

#### 92.4.1 1. Forums and Discussion

- r/MachineLearning

- Cross Validated
- AI Stack Exchange

### 92.4.2 2. Blogs and Newsletters

- Distill.pub
  - Interactive explanations
  - Visual learning
  - Deep insights

### 92.4.3 3. Tools and Libraries

- TensorFlow
- PyTorch
- Scikit-learn

## 92.5 Academic Papers

### 92.5.1 Foundational Papers

- "A Logical Calculus of Ideas Immanent in Nervous Activity" (McCulloch & Pitts, 1943)
- "Learning Internal Representations by Error Propagation" (Rumelhart et al., 1986)
- "Gradient-Based Learning Applied to Document Recognition" (LeCun et al., 1998)

### 92.5.2 Podcasts

- MLST: Machine Learning Street Talk
- Brainport/Iman interviews Sepp Hochreiter: XLSTM
- other podcasts from this series: 'Deep Dives with Iman'.
- Fontys AI Garage

### 92.5.3 Other

- email news letter: alphasignal.ai

# Part XII

# Tools-n-Technologies

# 93. My/Free AI tools

In this chapter I want to collect ideas for AI's that you can use (partly) for free, (mostly as in: free beer, as opposed to: free speech).

# 94. ChatGPT, Cursor.ai, Perplexity.ai, Claude, Gemini, Duck.ai

A lot of online tools give you some free tokens. how many tokens you get depends on of course what you are doing but also if if it's a search tool like Perplexity or a chatbot like ChatGPT, or a development environment like cursor.

# 95. chat.fontysict.nl

For Fontys ICT students who have an i-account, I advise looking at chat.fontysict.nl. Ask the project lead for an API token to be able to incorporate those models in your own apps.

# 96. Ollama (local or on your own server)

If you have a powerful laptop, then you could try running models locally, for example in Ollama. If you don't know, I advice you to just try it out. Start with a small model, like 'llame3.2' (2 GB) or 'phi3:latest' (2.4 GB)

# 97. Google Colab

In Google CoLab you can develop source code, for example a Python notebook, and you will have a copilot that will help you with programming.

# 98. Tools

- opencode

## 98.1 Agents

### 98.1.1 Agent Development Kit

- HF: Introduction to Agents
- https://google.github.io/adk-docs/https://google.github.io/adk-docs/

### 98.1.2 Open Agent Platform

- docs.oap.langchain.com

# 99. Codebook

Youtube/Shribe: How to create a codebook in qualitative research

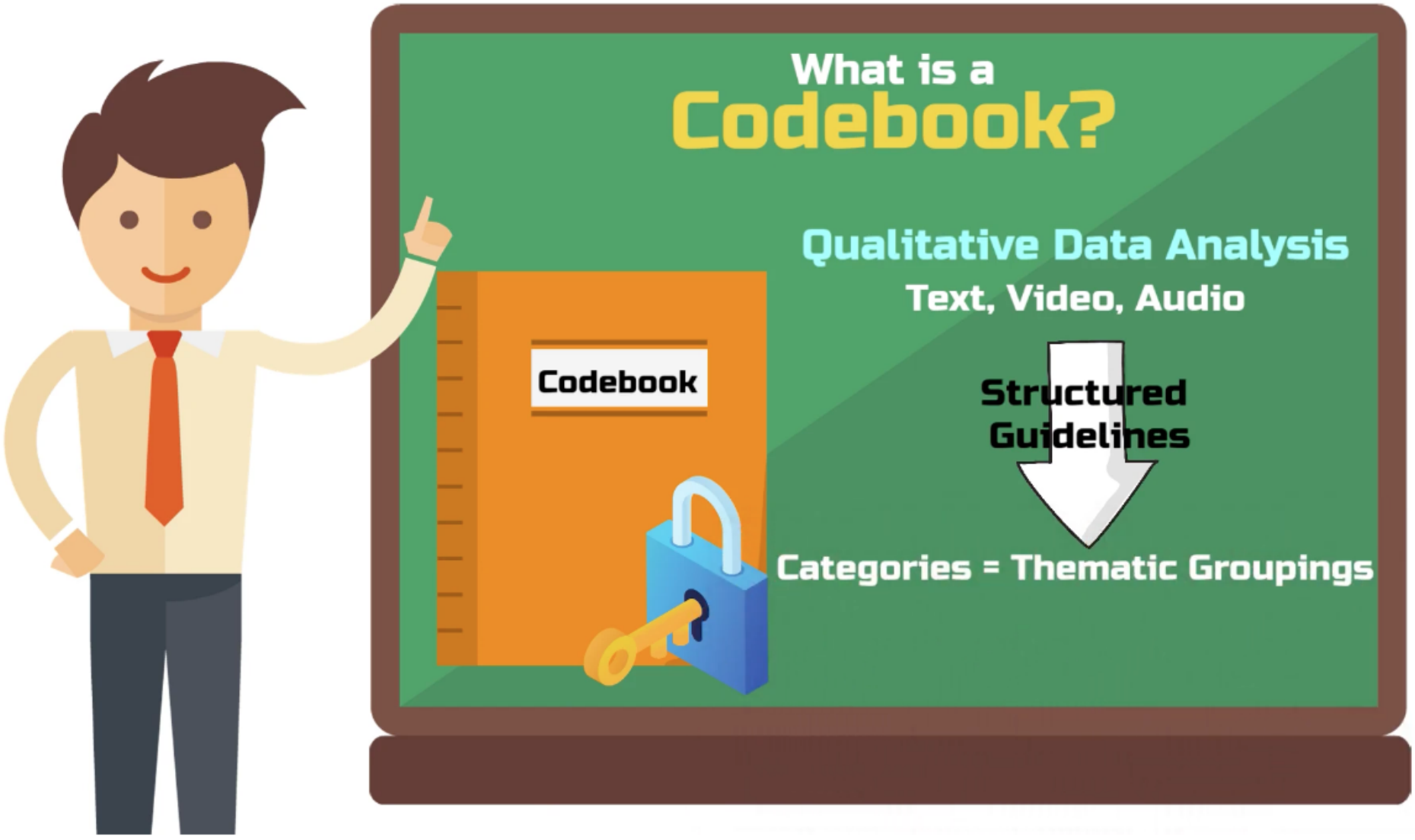



Figure 99.1: What is a Codebook?

# 100. Function-calling

A bridge between the 'non-strict' way that an LLM does things and the strict way that program code works: An LLM is train and instructed to recognize when and how to call a function/tool and 'create the call'.

deeplearning.ai short course: function-calling-and-data-extraction-with-llms

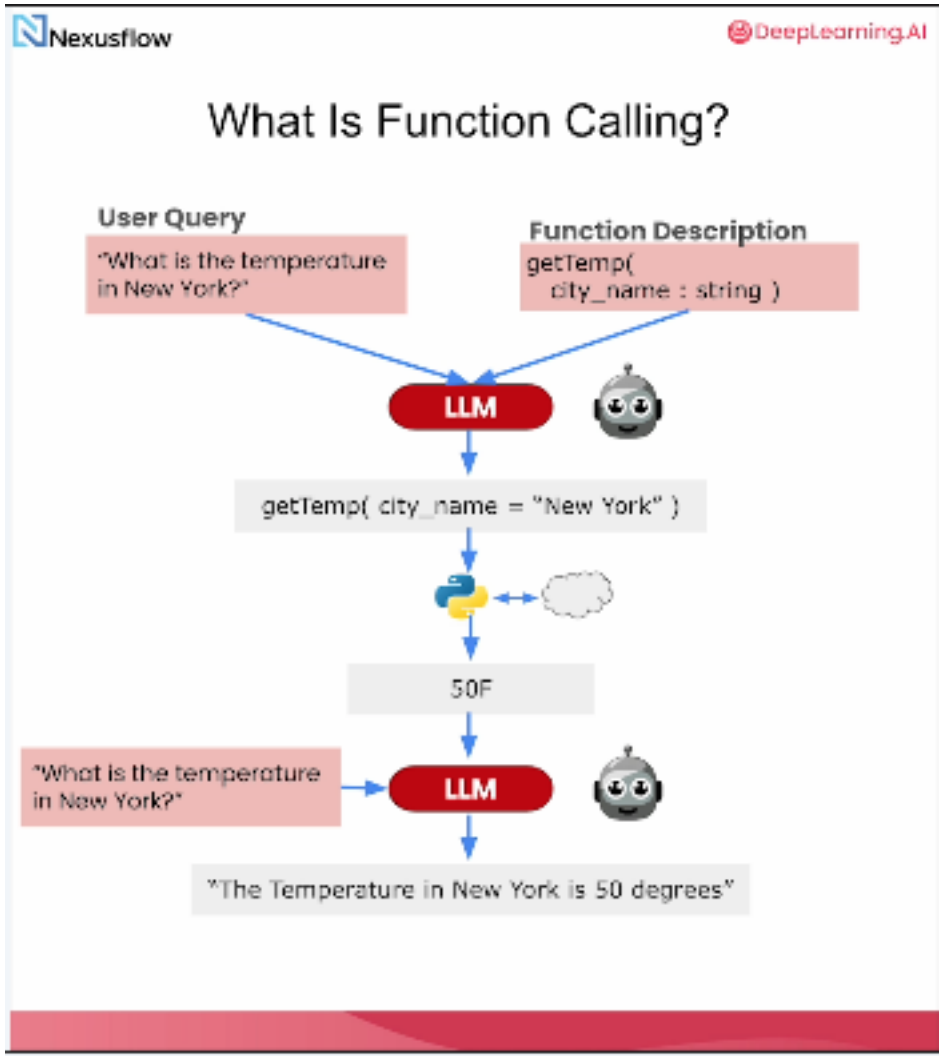



Figure 100.1: idea function-calling

# 101. Thunderbolt - Sovereign AI Stack

mozilla-introduces-thunderbolt

Mozilla Introduces Thunderbolt: An Enterprise AI Client Built for Control and Independence (2026-04)

"AI is too important to outsource. With Thunderbolt, we're giving organizations a sovereign AI client that allows them to decide how AI fits into their workflows — on their infrastructure, with their data, and on their terms." (Ryan Sipes, CEO of MZLA Technologies Corporation)




Figure 101.1: Thunderbolt

# 102. n8n (no/low code tool)

- n8n-workflows

# 103. Git / Version Control

One good reason to start using Git: Take control while Vibe Coding: use Git to exactly see and control changes.

Question: Is it still Vibe Coding when you take control?

**A video**

- Vibe Coding Course 7 – Version Control Basics (Git)

**Some learning material**

- Fontys ICT learning material
- Fontys ICT learning material - alternative
- great exercising site: learngitbranching.js.org
- learn git while playing minesweeper

**Some more possibly interesting videos**

- Cursor Vibe Coding Tutorial - For COMPLETE Beginners
- Let it cook - Vibe Coding with VS Code - Episode 1

# 104. Agents

- Engineering LLM-Based Agentic Systems
- HF: Introduction to Agents
- https://google.github.io/adk-docs/https://google.github.io/adk-docs/
- docs.oap.langchain.com

# 105. glaido

The voice layer for every app.

Uses local storage only, Nothing is saved to our database (so looks like it is still online!)

https://glaido.com/

# 106. Deepseek Harness - dsh

***Everything is a plug in.***

- What is a Harness?
- Creater Mode
- Cordis and Koishi
- Smalltalk: Everything is an Object
- Unix: Everything is a File
- Architecture
- youtube Deepseek just open-sourced Harness and it will blow your mind
- Julian Goldie: Deepseek Harness vs Hermes

Want Search with that? github.com/liustack/modsearch

# 107. MCP - Model Context Protocol

MCP is a standardization of the way to how LLM's connect to other tools.

Teacher r-huijts dove into it, created useful stuff:

- github.com/r-huijts

and there's more:

- short course: mcp-build-rich-context-ai-apps-with-anthropic
- short course: build-ai-apps-with-mcp-server-working-with-box-files
- MCP Quickstart
- modelcontextprotocol.info/
- Example Clients
- mcpservers.org
- Servers
- Greg Isenberg/Ras Mic explaining MCP
- short course MCP at deeplearning.ai
- Ruud mijn-nieuwe-mcp-server-laat-ai-zichzelf-actief tegenspreken
- mcp for whatsapp

# 108. MCP hands-on

Diving in…

So MCP standardizes the way I can combine sources of info (like RAG?) with an LLM.

Duckduckgoing for 'MCP vs ollama hands-on' (adding CLI afterwards) gives me some links to look at, and after a closer look these still seem interesting:

- agentic-rag-and-mcp
- Ollama MCP bridge
- ollama-mcp
- https://modelcontextprotocol.io/introduction
- lazy terminal
- https://apidog.com/blog/neovim-mcp-server/

…First I need an MCP client and an MCP server.

# 109. MCP Client

- 5ire looks nice.
- oterm

# 110. ACP - Agent Communication Protocol

To let Agents communicate, no matter what framework the Agents were built in.

- agentcommunicationprotocol.dev
- deeplearning.ai on ACP

# 111. herdr

Academind: herdr is a must use (10m)

Dave Ebbelaar: How to set up… (30m)

C. Lempa: A Better way to run terminal agents (22m)

# 112. paperreview.ai: Stanford Agentic Reviewer

- paperreview.ai: Stanford Agentic Reviewer

# Part XIII

# Experiments

# 113. Experiments

Experiments, maybe incomplete… never finished, the whole reutemeteut!